INSTITUTO TECNOLOGICO JOSÉ MARIO MOLINA PASQUEL Y HENRIQUEZ

UNIDAD ACADEMICA ZAPOTLANEJO

Proyecto: Manual de prácticas Desarrollo de Aplicaciones para dispositivos móviles



INSTITUTO TECNOLÓGICO SUPERIOR
ZAPOTLANEJO
INNOVACIÓN • TECNOLOGÍA • FUTURO

Alumno: Oscar Alejandro Alvarez Aceves

Carrera: Ingeniería en informática

Materia: Desarrollo de Aplicaciones para dispositivos móviles

Docente: Osvaldo Rene Rojo Roa

Contenido

UNIDAD 1 3
Practica 1 “Hola mundo” 3
Practica 2 “Contenedores, Children, botones y cuadros de texto” 6
Practica 3 “Suma” 9
Practica 4 “Operaciones aritméticas” 13
Practica 4.1 “Operaciones aritméticas con menú desplegable” 17
Practica 5 “Imágenes” 21
Practica 6 “envió de información entre pantallas” 24
Unidad 2 30
Practica 7 “Cambio de pantallas y envió de datos” 30
Practica 8 “Listas y conteo de elementos” 32
Practica 9 “Listas de números, conteo de números par e impar” 36
Practica 10 ‘’Login, validación de datos, envió de información y lista’’ 43
Practica derecho a examen 52



UNIDAD 1

Practica 1 “Hola mundo”

En esta práctica y utilizando las diferentes clases de flutter logramos hacer una aplicación llamada práctica uno que mostrara el texto en pantalla hola mundo también utilizamos métodos como el columna Para poder centrar nuestro texto dentro de la pantalla de nuestro dispositivo

```dart
import 'package:flutter/material.dart';
import 'package:practica1/widget/inicio.dart';

// import 'package:practica1/widget2/Practica2.dart';
// import 'package:practica1/operaciones/suma.dart';
//import 'package:practica1/aritmeticas/practica4.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Inicio());
  }
}
```

Ilustración 1 Código main



```dart
import 'package:flutter/material.dart';

// Importa la biblioteca Material de Flutter, que contiene los widgets
// y componentes necesarios para crear la interfaz gráfica.
class Inicio extends StatefulWidget {
  // Declara la clase Inicio como un StatefulWidget.
  // Esto permite que la pantalla pueda cambiar su estado durante la ejecución.
  const Inicio({super.key});
  // Constructor de la clase Inicio.
  // super.key permite identificar este widget dentro del árbol de widgets.
  @override
  State<StatefulWidget> createState() {
    // Método encargado de crear y asociar el estado del widget Inicio.
    return Disenio();
    // Retorna una instancia de la clase Disenio,
    // que contiene la estructura y diseño de la pantalla.
  }
}

class Disenio extends State<Inicio> {
  // Clase que representa el estado de Inicio.
  // Aquí se construye la interfaz que se mostrará en pantalla.
  @override
  Widget build(BuildContext context) {
    // Método build que construye y devuelve la interfaz gráfica.
    // context contiene información sobre la ubicación del widget dentro
    // del árbol de widgets de Flutter.
    return Scaffold(
      // Scaffold proporciona la estructura básica de una pantalla,
      // incluyendo elementos como AppBar y body.
      appBar: AppBar(
        // Crea la barra superior de la aplicación.
        title: Text('Practica 1', style: TextStyle(color: Colors.white)),
        // Muestra el título "Practica 1" en la barra superior.
        // El texto se establece en color blanco.
        backgroundColor: const Color.fromARGB(255, 6, 79, 238),
        // Establece el color de fondo de la barra superior
        // utilizando valores RGB y una opacidad de 255.
      ),
      body: Column(
        // Column organiza sus elementos hijos de manera vertical.
        mainAxisAlignment: MainAxisAlignment.center,
        // Centra verticalmente los elementos que se encuentran dentro
        // de la columna.
        children: [
          // Lista de widgets que se mostrarán dentro de la columna.
          Text(
            'Hola mundo',
            // Primer texto que se mostrará en la pantalla.
            style: TextStyle(color: Colors.deepPurpleAccent, fontSize: 25),
            // Define el estilo del texto:
            // color morado y tamaño de fuente de 25.
          ),
          Text(
            'Hola',
            // Segundo texto que se mostrará debajo del primer texto.
            style: TextStyle(color: Colors.deepPurpleAccent, fontSize: 25),
            // Aplica el mismo estilo al segundo texto:
            // color morado y tamaño de fuente de 25.
          ),
        ], // Finaliza la lista de elementos de la Column.
      ), // Finaliza el contenido principal de la pantalla.
    ); // Finaliza el Scaffold.
  } // Finaliza el método build.
} // Finaliza la clase Disenio.

```

Ilustración 2 Código Practica 1





Ilustración 3 Ejecución practica 1

Practica 2 “Contenedores, Children, botones y cuadros de texto”

En esta práctica utilizando los diferentes métodos que tales como children, child, padding, cuadros de texto y Input decoration podemos crear de manera más visual una aplicación que nos permitía interactuar y escribir un nombre dentro de ella

```dart
import 'package:flutter/material.dart';
//import 'package:practica1/widget/inicio.dart';
import 'package:practica1/widget2/Practica2.dart';

// import 'package:practica1/operaciones/suma.dart';
//import 'package:practica1/aritmeticas/practica4.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Practica2());
  }
}

```

Ilustración 4 Código main



```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Material Design de Flutter.
class Practica2 extends StatefulWidget { // Declara la pantalla Practica2 como un StatefulWidget.
  const Practica2({super.key}); // Constructor de la clase Practica2.
  @override
  State<StatefulWidget> createState() { // Crea el estado que manejará la pantalla.
    return Disenio(); // Retorna la clase que contiene el diseño de la pantalla.
  }
}
class Disenio extends State<Practica2> { // Define el estado y diseño de la pantalla Practica2.
  @override
  Widget build(BuildContext context) { // Construye la interfaz gráfica de la pantalla.
    return Scaffold( // Proporciona la estructura principal de la pantalla.
      appBar: AppBar( // Crea la barra superior de la aplicación.
        title: Text('Practica 2', style: TextStyle(color: Colors.white)), // Muestra el título de la práctica en color blanco.
        backgroundColor: const Color.fromARGB(255, 6, 79, 238), // Establece el color azul de la barra superior.
      ),
      //contenedores // Indica que en esta sección se utilizan diferentes contenedores.
      body: Column( // Organiza los elementos de la pantalla de forma vertical.
        mainAxisAlignment: MainAxisAlignment.center, // Centra los elementos verticalmente.
        children: [ // Contiene todos los elementos que se mostrarán en la columna.
          Container( // Crea un contenedor para el primer texto.
            color: Colors.black, // Establece el fondo del contenedor en color negro.
            width: double.infinity, // Hace que el contenedor ocupe todo el ancho disponible.
            child: Text( // Agrega un texto dentro del contenedor.
              'Hola mundo', // Texto que se mostrará.
              textAlign: TextAlign.center, // Centra el texto horizontalmente.
              style: TextStyle( // Define las caracteristicas visuales del texto.
                color: Colors.white, // Establece el texto en color blanco.
                fontFamily: "Times New Roman", // Define la fuente Times New Roman.
                fontSize: 24, // Establece el tamaño de la letra en 24.
              ),
            ),
          ),
          SizedBox(height: 10), // Agrega un espacio vertical de 10 píxeles.
          Container( // Crea un segundo contenedor.
            padding: EdgeInsets.all(25), // Agrega un espacio interno de 25 píxeles en todos los lados.
            color: Colors.red, // Establece el fondo del contenedor en color rojo.
            width: double.infinity, // Hace que el contenedor ocupe todo el ancho disponible.
            child: Text( // Agrega un texto dentro del contenedor.
              'Hola', // Texto que se mostrará.
              textAlign: TextAlign.center, // Centra el texto horizontalmente.
              style: TextStyle(color: Colors.white, fontSize: 24), // Define el color blanco y tamaño 24 del texto.
            ),
          ),
          SizedBox(height: 10), // Agrega un espacio vertical de 10 píxeles.
          SizedBox( // Crea un espacio que también permite controlar el tamaño del botón.
            width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
            child: ElevatedButton(onPressed: () {}, child: Text('Boton ')), // Crea un botón que actualmente no ejecuta ninguna acción.
          ),
          SizedBox(height: 10), // Agrega un espacio vertical de 10 píxeles.
          TextField( // Crea un campo donde el usuario puede introducir texto.
            decoration: InputDecoration(labelText: 'Escribe el nombre'), // Muestra la etiqueta "Escribe el nombre" en el campo.
          ),
        ],
      ),
    );
  }
}

```

Ilustración 5 Código Practica 2

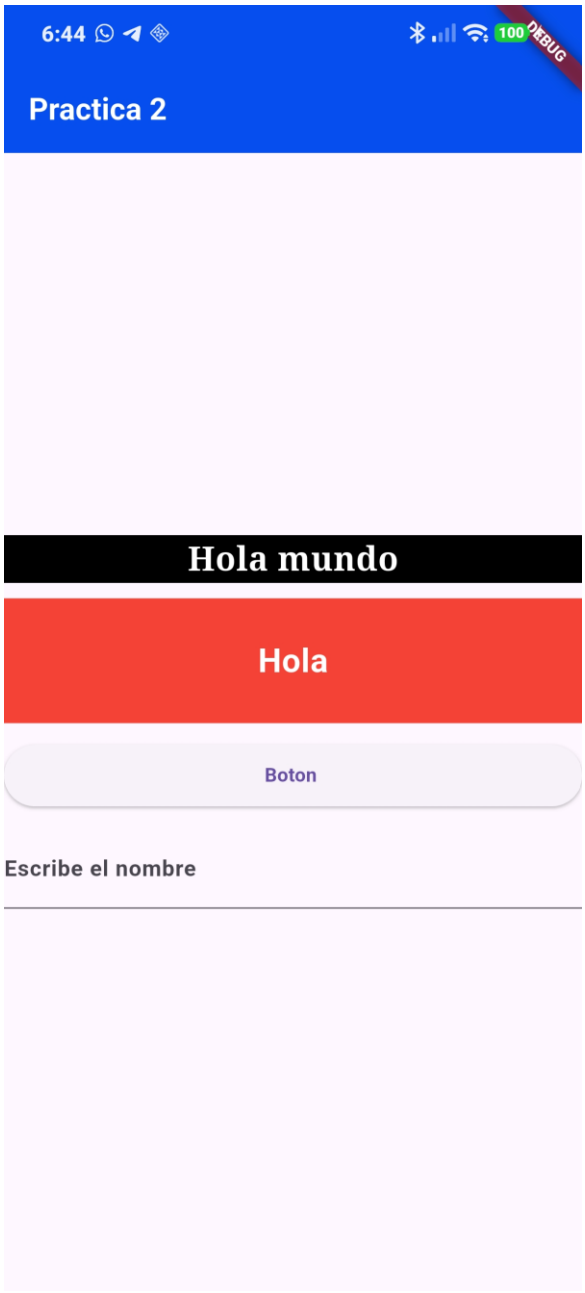


Ilustración 6 Ejecución Practica 2

Practica 3 “Suma”

En esta práctica utilizamos 2 cuadros de texto para pedir al usuario que ingrese 2 números aleatorios para poder realizar una suma entre ellos y mostrar el resultado la suma se calcula a través de un método asignado un botón mostrando el resultado en un texto en la pantalla si el usuario ingresara cualquier otro valor lo convierte en un cero para evitar que la aplicación falle

```dart
import 'package:flutter/material.dart';
//import 'package:practica1/widget/inicio.dart';
//import 'package:practica1/widget2/Practica2.dart';
import 'package:practica1/operaciones/suma.dart';

//import 'package:practica1/aritmeticas/practica4.dart';

void main() {
  runApp(const MyApp());
}

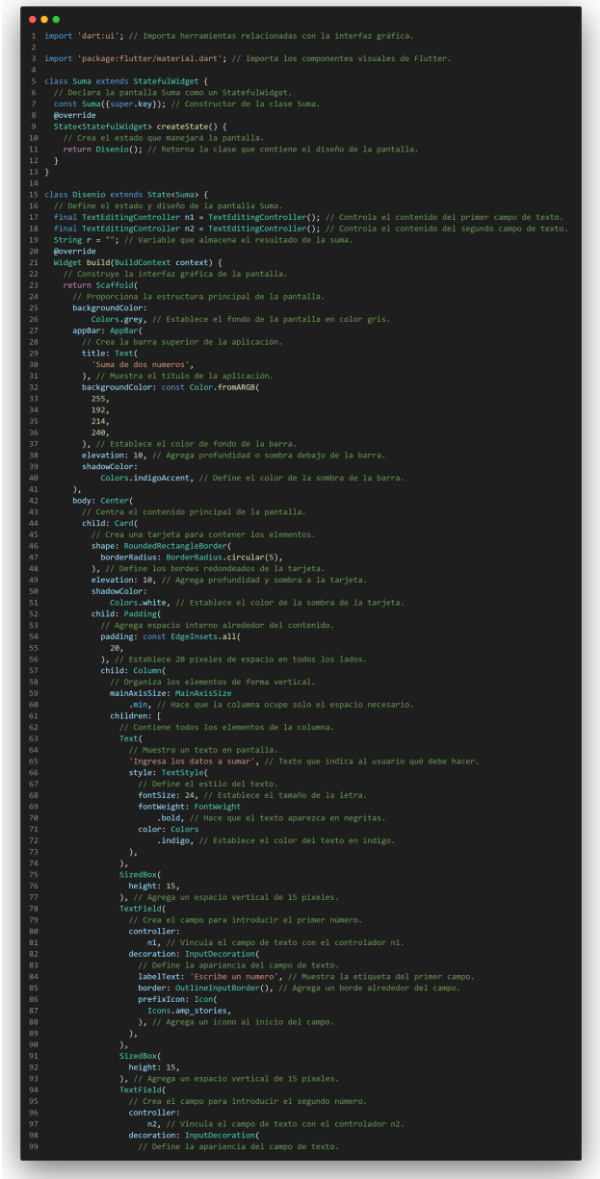
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Suma());
  }
}

```

Ilustración 7 Código Practica 3





```dart
import 'dart:ui'; // Importa herramientas relacionadas con la interfaz gráfica.

import 'package:flutter/material.dart'; // Importa los componentes visuales de Flutter.

class Suma extends StatefulWidget {
  // Declara la pantalla Suma como un StatefulWidget.
  const Suma({super.key}); // Constructor de la clase Suma.
  @override
  State<StatefulWidget> createState() {
    // Crea el estado que manejará la pantalla.
    return Disenio(); // Retorna la clase que contiene el diseño de la pantalla.
  }
}

class Disenio extends State<Suma> {
  // Define el estado y diseño de la pantalla Suma.
  final TextEditingController n1 = TextEditingController(); // Controla el contenido del primer campo de texto.
  final TextEditingController n2 = TextEditingController(); // Controla el contenido del segundo campo de texto.
  String r = ""; // Variable que almacena el resultado de la suma.
  @override
  Widget build(BuildContext context) {
    // Construye la interfaz gráfica de la pantalla.
    return Scaffold(
      // Proporciona la estructura principal de la pantalla.
      backgroundColor:
          Colors.grey, // Establece el fondo de la pantalla en color gris.
      appBar: AppBar(
        // Crea la barra superior de la aplicación.
        title: Text(
          'Suma de dos numeros',
        ), // Muestra el título de la aplicación.
        backgroundColor: const Color.fromARGB(
          255,
          192,
          214,
          240,
        ), // Establece el color de fondo de la barra.
        elevation: 10, // Agrega profundidad o sombra debajo de la barra.
        shadowColor:
            Colors.indigoAccent, // Define el color de la sombra de la barra.
      ),
      body: Center(
        // Centra el contenido principal de la pantalla.
        child: Card(
          // Crea una tarjeta para contener los elementos.
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(5),
          ), // Define los bordes redondeados de la tarjeta.
          elevation: 10, // Agrega profundidad y sombra a la tarjeta.
          shadowColor:
              Colors.white, // Establece el color de la sombra de la tarjeta.
          child: Padding(
            // Agrega espacio interno alrededor del contenido.
            padding: const EdgeInsets.all(
              20,
            ), // Establece 20 píxeles de espacio en todos los lados.
            child: Column(
              // Organiza los elementos de forma vertical.
              mainAxisSize: MainAxisSize
                  .min, // Hace que la columna ocupe solo el espacio necesario.
              children: [
                // Contiene todos los elementos de la columna.
                Text(
                  // Muestra un texto en pantalla.
                  'Ingresa los datos a sumar', // Texto que indica al usuario qué debe hacer.
                  style: TextStyle(
                    // Define el estilo del texto.
                    fontSize: 24, // Establece el tamaño de la letra.
                    fontWeight: FontWeight
                        .bold, // Hace que el texto aparezca en negritas.
                    color: Colors
                        .indigo, // Establece el color del texto en índigo.
                  ),
                ),
                SizedBox(
                  height: 15,
                ), // Agrega un espacio vertical de 15 píxeles.
                TextField(
                  // Crea el campo para introducir el primer número.
                  controller:
                      n1, // Vincula el campo de texto con el controlador n1.
                  decoration: InputDecoration(
                    // Define la apariencia del campo de texto.
                    labelText: 'Escribe un numero', // Muestra la etiqueta del primer campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(
                      Icons.amp_stories,
                    ), // Agrega un ícono al inicio del campo.
                  ),
                ),
                SizedBox(
                  height: 15,
                ), // Agrega un espacio vertical de 15 píxeles.
                TextField(
                  // Crea el campo para introducir el segundo número.
                  controller:
                      n2, // Vincula el campo de texto con el controlador n2.
                  decoration: InputDecoration(
                    // Define la apariencia del campo de texto.
```

Ilustración 8 Codigo Practica 3





```
                    labelText: 'Escribe otro numero', // Muestra la etiqueta del segundo campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(
                      Icons.amp_stories,
                    ), // Agrega un icono al inicio del campo.
                  ),
                ),
                SizedBox(
                  height: 20,
                ), // Agrega un espacio vertical de 20 pixeles.
                Text(
                  // Muestra el resultado de la operación.
                  r, // Utiliza el contenido de la variable r como texto.
                  style: TextStyle(
                    // Define el estilo del resultado.
                    fontSize: 24, // Establece el tamaño de la letra.
                    fontWeight:
                        FontWeight.bold, // Muestra el resultado en negritas.
                    color: Colors.indigo, // Establece el color del resultado.
                  ),
                ),
                SizedBox(
                  height: 20,
                ), // Agrega un espacio vertical de 20 pixeles.
                SizedBox(
                  // Permite controlar el tamaño del botón.
                  width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                  child: ElevatedButton.icon(
                    // Crea un botón elevado con un icono.
                    icon: Icon(Icons.summarize), // Agrega el icono al botón.
                    onPressed: () {
                      // Define la acción que se ejecuta al presionar el botón.
                      final int a = int.tryParse(n1.text) ?? 0; // Convierte el primer valor de texto a entero; si falla, utiliza 0.
                      final int b = int.tryParse(n2.text) ?? 0; // Convierte el segundo valor de texto a entero; si falla, utiliza 0.
                      setState(() {
                        // Actualiza la interfaz después de realizar la operación.
                        r = "${a + b}"; // Suma los dos números y guarda el resultado en r.
                      });
                    },
                    label: Text(
                      // Define el texto que aparecerá dentro del botón.
                      'Sumar', // Texto mostrado en el botón.
                      style: TextStyle(
                        // Define el estilo del texto del botón.
                        fontSize: 24, // Establece el tamaño de la letra.
                        fontWeight: FontWeight
                            .bold, // Hace que el texto aparezca en negritas.
                        color: Colors.indigo, // Establece el color del texto.
                      ),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
 }
}
```

Ilustración 9 Codigo Practica 3



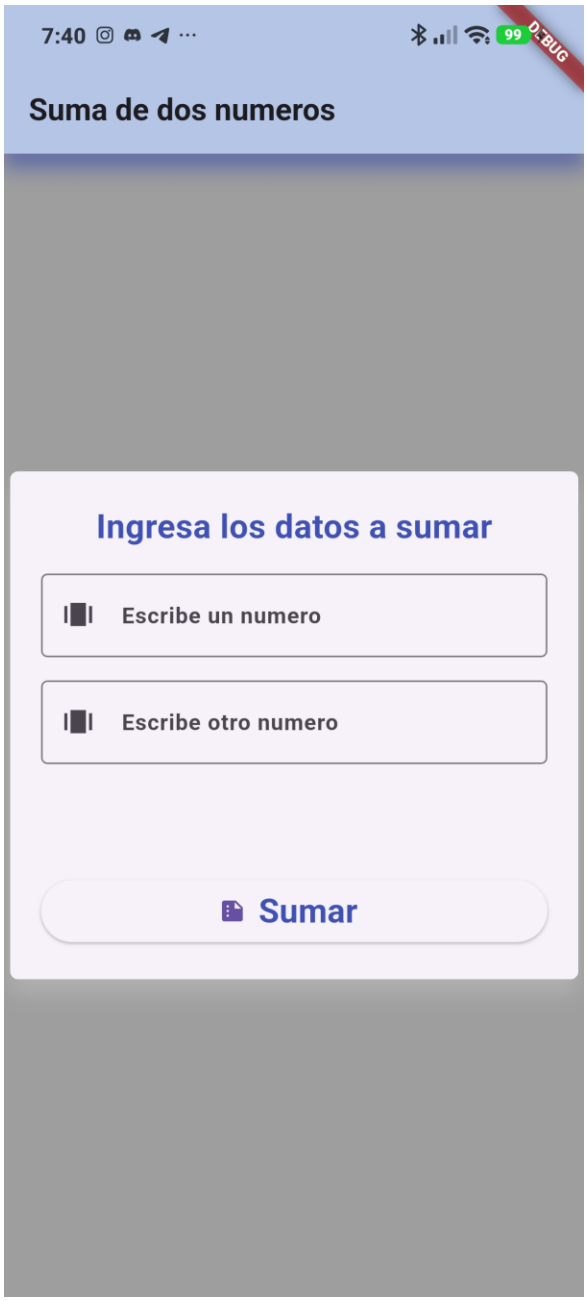


Ilustración 10 Ejecución practica 3

Practica 4 “Operaciones aritméticas”

En estas prácticas desarrollamos una aplicación que sea capaz de realizar las operaciones básicas aritméticas utilizando métodos específicos asignados a cada tipo de operación representadas por un botón en la pantalla y los datos siendo ingresados por cuadros de texto por el usuario

```dart
import 'package:flutter/material.dart';
//import 'package:practica1/widget/inicio.dart';
//import 'package:practica1/widget2/Practica2.dart';
//import 'package:practica1/operaciones/suma.dart';

import 'package:practica1/aritmeticas/practica4.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: aritmetica());
  }
}

```

Ilustración 11 Código Practica 4

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Flutter.
class aritmetica extends StatefulWidget { // Declara la pantalla aritmetica como un StatefulWidget.
  const aritmetica({super.key}); // Constructor de la clase aritmetica.
  @override
  State<StatefulWidget> createState() { // Crea el estado que manejará la pantalla.
    return Disenio(); // Retorna la clase que contiene el diseño de la pantalla.
  }
}
class Disenio extends State<aritmetica> { // Define el estado y diseño de la pantalla aritmetica.
  final TextEditingController n1 = TextEditingController(); // Controla el contenido del primer campo de texto.
  final TextEditingController n2 = TextEditingController(); // Controla el contenido del segundo campo de texto.
  String r = ""; // Variable que almacena el resultado de la operación.
  @override
  Widget build(BuildContext context) { // Construye la interfaz gráfica de la pantalla.
    return Scaffold( // Proporciona la estructura principal de la pantalla.
      backgroundColor: Colors.blueGrey, // Establece el fondo de la pantalla en color azul grisáceo.
      appBar: AppBar( // Crea la barra superior de la aplicación.
        title: Text('Operaciones Aritmeticas'), // Muestra el título de la pantalla.
        backgroundColor: const Color.fromARGB(255, 192, 214, 240), // Establece el color de fondo de la barra.
        elevation: 10, // Agrega profundidad o sombra debajo de la barra.
        shadowColor: Colors.indigoAccent, // Define el color de la sombra de la barra.
      ),
      body: Center( // Centra el contenido principal de la pantalla.
        child: Card( // Crea una tarjeta para contener los elementos.
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(5)), // Define los bordes redondeados de la tarjeta.
          elevation: 10, // Agrega profundidad y sombra a la tarjeta.
          shadowColor: Colors.white, // Establece el color de la sombra de la tarjeta.
          child: Padding( // Agrega espacio interno alrededor del contenido.
            padding: const EdgeInsets.all(20), // Establece 20 píxeles de espacio en todos los lados.
            child: Column( // Organiza los elementos de forma vertical.
              mainAxisSize: MainAxisSize.min, // Hace que la columna ocupe solo el espacio necesario.
              children: [ // Contiene todos los elementos de la columna.
                Text( // Muestra un texto en pantalla.
                  'Ingresa los datos a sumar', // Indica al usuario que debe ingresar los números.
                  style: TextStyle( // Define el estilo del texto.
                    fontSize: 24, // Establece el tamaño de la letra.
                    fontWeight: FontWeight.bold, // Hace que el texto aparezca en negritas.
                    color: Colors.indigo, // Establece el color del texto.
                  ),
                ),
                SizedBox(height: 15), // Agrega un espacio vertical de 15 píxeles.
                TextField( // Crea el campo para introducir el primer número.
                  controller: n1, // Vincula el campo de texto con el controlador n1.
                  decoration: InputDecoration( // Define la apariencia del campo de texto.
                    labelText: 'Escribe un numero', // Muestra la etiqueta del primer campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(Icons.amp_stories), // Agrega un icono al inicio del campo.
                  ),
                ),
                SizedBox(height: 15), // Agrega un espacio vertical de 15 píxeles.
                TextField( // Crea el campo para introducir el segundo número.
                  controller: n2, // Vincula el campo de texto con el controlador n2.
                  decoration: InputDecoration( // Define la apariencia del campo de texto.
                    labelText: 'Escribe otro numero', // Muestra la etiqueta del segundo campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(Icons.amp_stories), // Agrega un icono al inicio del campo.
                  ),
                ),
                SizedBox(height: 20), // Agrega un espacio vertical de 20 píxeles.
                Text( // Muestra el resultado de la operación.
                  r, // Utiliza el contenido de la variable r como resultado.
                  style: TextStyle( // Define el estilo del resultado.
                    fontSize: 24, // Establece el tamaño de la letra.
                    fontWeight: FontWeight.bold, // Muestra el resultado en negritas.
                    color: Colors.indigo, // Establece el color del resultado.
                  ),
                ),
                SizedBox(height: 20), // Agrega un espacio vertical de 20 píxeles.

                //Suma // Indica que el siguiente botón realiza la suma.
                SizedBox( // Permite controlar el tamaño del botón.
                  width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                  child: ElevatedButton.icon( // Crea un botón elevado con un icono.
                    icon: Icon(Icons.summarize), // Agrega un icono al botón.
                    onPressed: () { // Define la acción que se ejecuta al presionar el botón.
                      final int a = int.tryParse(n1.text) ?? 0; // Convierte el primer valor a entero; si falla, utiliza 0.
                      final int b = int.tryParse(n2.text) ?? 0; // Convierte el segundo valor a entero; si falla, utiliza 0.
                      setState(() { // Actualiza la interfaz después de realizar la operación.
                        r = "${a + b}"; // Suma los dos números y guarda el resultado en r.
                      });
                    },
                    label: Text( // Define el texto que aparecerá dentro del botón.
                      'Sumar', // Texto mostrado en el botón.
                      style: TextStyle( // Define el estilo del texto.
                        fontSize: 24, // Establece el tamaño de la letra.
                        fontWeight: FontWeight.bold, // Hace que el texto aparezca en negritas.
                        color: Colors.indigo, // Establece el color del texto.
                      ),
                    ),
                  ),
                ),
              ),

```

Ilustración 12 Código Practica 4



```dart

              SizedBox(height: 20), // Agrega un espacio vertical de 20 píxeles.
              SizedBox( // Permite controlar el tamaño del botón.
                width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                child: ElevatedButton.icon( // Crea un botón elevado con un ícono.
                  icon: Icon(Icons.summarize), // Agrega un ícono al botón.
                  onPressed: () { // Define la acción que se ejecuta al presionar el botón.
                    final int a = int.tryParse(n1.text) ?? 0; // Convierte el primer valor a entero; si falla, utiliza 0.
                    final int b = int.tryParse(n2.text) ?? 0; // Convierte el segundo valor a entero; si falla, utiliza 0.
                    setState(() { // Actualiza la interfaz después de realizar la operación.
                      r = "${a - b}"; // Resta el segundo número al primero y guarda el resultado en r.
                    });
                  },
                  label: Text( // Define el texto que aparecerá dentro del botón.
                    'Restar', // Texto mostrado en el botón.
                    style: TextStyle( // Define el estilo del texto.
                      fontSize: 24, // Establece el tamaño de la letra.
                      fontWeight: FontWeight.bold, // Hace que el texto aparezca en negritas.
                      color: Colors.indigo, // Establece el color del texto.
                    ),
                  ),
                ),
              ),
              SizedBox(height: 20), // Agrega un espacio vertical de 20 píxeles.
              SizedBox( // Permite controlar el tamaño del botón.
                width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                child: ElevatedButton.icon( // Crea un botón elevado con un ícono.
                  icon: Icon(Icons.summarize), // Agrega un ícono al botón.
                  onPressed: () { // Define la acción que se ejecuta al presionar el botón.
                    final int a = int.tryParse(n1.text) ?? 0; // Convierte el primer valor a entero; si falla, utiliza 0.
                    final int b = int.tryParse(n2.text) ?? 0; // Convierte el segundo valor a entero; si falla, utiliza 0.
                    setState(() { // Actualiza la interfaz después de realizar la operación.
                      r = "${a * b}"; // Multiplica los dos números y guarda el resultado en r.
                    });
                  },
                  label: Text( // Define el texto que aparecerá dentro del botón.
                    'Multiplicar', // Texto mostrado en el botón.
                    style: TextStyle( // Define el estilo del texto.
                      fontSize: 24, // Establece el tamaño de la letra.
                      fontWeight: FontWeight.bold, // Hace que el texto aparezca en negritas.
                      color: Colors.indigo, // Establece el color del texto.
                    ),
                  ),
                ),
              ),
              SizedBox(height: 20), // Agrega un espacio vertical de 20 píxeles.
              SizedBox( // Permite controlar el tamaño del botón.
                width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                child: ElevatedButton.icon( // Crea un botón elevado con un ícono.
                  icon: Icon(Icons.summarize), // Agrega un ícono al botón.
                  onPressed: () { // Define la acción que se ejecuta al presionar el botón.
                    final int a = int.tryParse(n1.text) ?? 0; // Convierte el primer valor a entero; si falla, utiliza 0.
                    final int b = int.tryParse(n2.text) ?? 0; // Convierte el segundo valor a entero; si falla, utiliza 0.
                    setState(() { // Actualiza la interfaz después de realizar la operación.
                      r = "${a / b}"; // Divide el primer número entre el segundo y guarda el resultado en r.
                    });
                  },
                  label: Text( // Define el texto que aparecerá dentro del botón.
                    'Dividir', // Texto mostrado en el botón.
                    style: TextStyle( // Define el estilo del texto.
                      fontSize: 24, // Establece el tamaño de la letra.
                      fontWeight: FontWeight.bold, // Hace que el texto aparezca en negritas.
                      color: Colors.indigo, // Establece el color del texto.
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

```

Ilustración 13 Código Practica 4



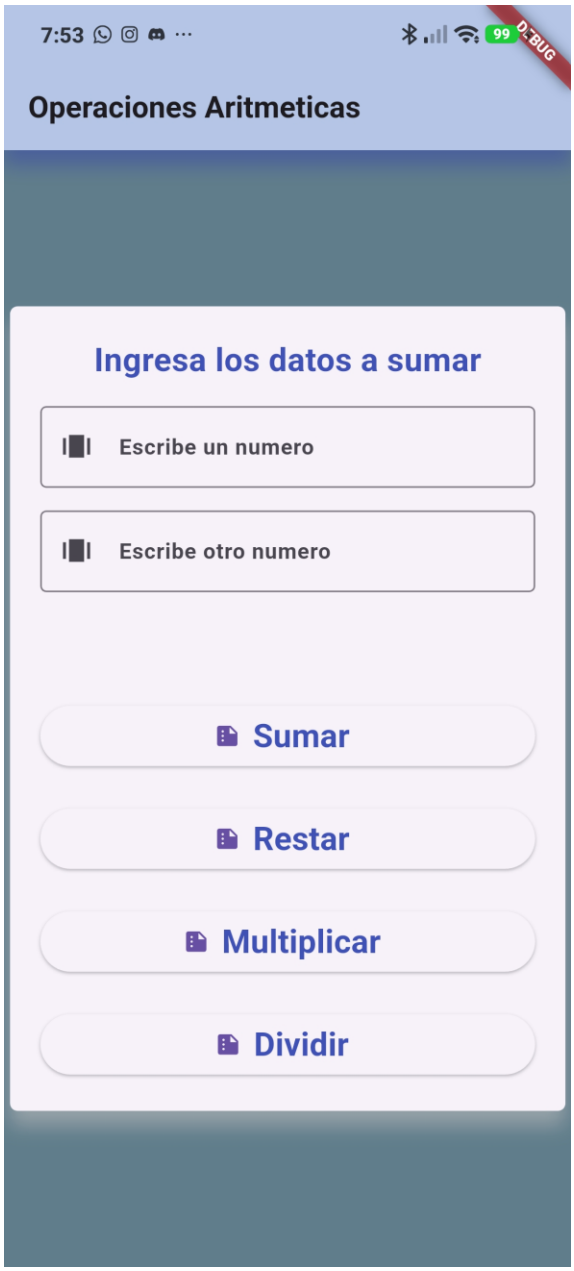


Ilustración 14 ejecución Practica 4

Practica 4.1 “Operaciones aritméticas con menú desplegable”

```dart
import 'package:flutter/material.dart';
//import 'package:practica1/widget/inicio.dart';
//import 'package:practica1/widget2/Practica2.dart';
//import 'package:practica1/operaciones/suma.dart';
import 'package:practica1/aritmeticas/practica4-1.dart';

//import 'package:practica1/aritmeticas/practica4.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Programa4());
  }
}

```

Ilustración 15 Código practica 4.1

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Flutter.

class Programa4 extends StatefulWidget {
  // Declara Programa4 como un StatefulWidget.
  @override
  State<StatefulWidget> createState() {
    // Crea el estado que manejará la pantalla.
    return Clases(); // Retorna la clase que contiene la lógica y diseño.
  }
}

class Clases extends State<Programa4> {
  // Define el estado de la pantalla Programa4.
  final TextEditingController n1 = TextEditingController(); // Controla el contenido del primer campo de texto.
  final TextEditingController n2 = TextEditingController(); // Controla el contenido del segundo campo de texto.
  String
  r = "",
  oper =
      ""; // r almacena el resultado y oper almacena la operación seleccionada.
  void Operaciones() {
    // Método que realiza la operación seleccionada.
    final int a =
        int.tryParse(n1.text) ??
        0; // Convierte el primer valor de texto a entero; si falla, utiliza 0.
    final int b =
        int.tryParse(n2.text) ??
        0; // Convierte el segundo valor de texto a entero; si falla, utiliza 0.
    setState(() {
      // Actualiza la interfaz después de realizar la operación.
      print(
        "Oper: " + oper,
      ); // Muestra en la consola la operación seleccionada.
      if (oper == "Suma") {
        // Comprueba si la operación seleccionada es una suma.
        r = "${a + b}"; // Suma los dos números y guarda el resultado.
      }
      if (oper == "Resta") {
        // Comprueba si la operación seleccionada es una resta.
        r = "${a - b}"; // Resta el segundo número al primero y guarda el resultado.
      }
      if (oper == "Multiplicación") {
        // Comprueba si la operación seleccionada es una multiplicación.
        r = "${a * b}"; // Multiplica los dos números y guarda el resultado.
      }
      if (oper == "División") {
        // Comprueba si la operación seleccionada es una división.
        r = "${a / b}"; // Divide el primer número entre el segundo y guarda el resultado.
      }
      ScaffoldMessenger.of(context).showSnackBar(
        // Permite mostrar un mensaje temporal en la pantalla.
        SnackBar(
          content: Text("El resultado:" + r),
        ), // Muestra el resultado de la operación en un SnackBar.
      );
    });
  }

  @override
  Widget build(BuildContext context) {
    // Construye la interfaz gráfica de la pantalla.
    return Scaffold(
      // Proporciona la estructura principal de la pantalla.
      backgroundColor:
          Colors.grey, // Establece el fondo de la pantalla en color gris.
      appBar: AppBar(
        // Crea la barra superior de la aplicación.
        title: Text('Suma de dos numeros'), // Muestra el título de la pantalla.
        backgroundColor: const Color.fromARGB(
          255,
          192,
          214,
          240,
        ), // Establece el color de fondo de la barra.
        elevation: 10, // Agrega profundidad o sombra debajo de la barra.
        shadowColor:
            Colors.indigoAccent, // Define el color de la sombra de la barra.
      ),
      body: Center(
        // Centra el contenido principal de la pantalla.
        child: Card(
          // Crea una tarjeta para contener los elementos.
          shape: RoundedRectangleBorder(
            // Define la forma de los bordes de la tarjeta.
            borderRadius: BorderRadius.circular(
              5,
            ), // Establece bordes redondeados de 5 píxeles.
          ),
          elevation: 15, // Agrega profundidad y sombra a la tarjeta.
          shadowColor:
              Colors.white, // Establece el color de la sombra de la tarjeta.
          child: Padding(
            // Agrega espacio interno alrededor del contenido.
            padding: EdgeInsets.all(
              20,
            ), // Establece 20 píxeles de espacio en todos los lados.
            child: Column(
              // Organiza los elementos de forma vertical.
              mainAxisSize: MainAxisSize
                  .min, // Hace que la columna ocupe solo el espacio necesario.
              children: [
```

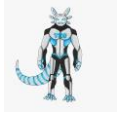
Ilustración 16 Código practica 4.1



```
                // Contiene todos los elementos de la columna.
                Text(
                  'Ingresa los datos a sumar', // Muestra un mensaje para indicar que se deben ingresar los números.
                  style: TextStyle(
                    // Define el estilo del texto.
                    fontSize: 24, // Establece el tamaño de la letra.
                    fontWeight: FontWeight
                        .bold, // Hace que el texto aparezca en negritas.
                    color: Colors.indigo, // Establece el color del texto.
                  ),
                ),
                SizedBox(
                  height: 15,
                ), // Agrega un espacio vertical de 15 píxeles.
                TextField(
                  // Crea el campo para introducir el primer número.
                  controller: n1, // Vincula el campo con el controlador n1.
                  decoration: InputDecoration(
                    // Define la apariencia del campo de texto.
                    labelText: "Escribe un numero", // Muestra la etiqueta del primer campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(
                      Icons.amp_stories,
                    ), // Agrega un icono al inicio del campo.
                  ),
                ),
                SizedBox(
                  height: 15,
                ), // Agrega un espacio vertical de 15 píxeles.
                TextField(
                  // Crea el campo para introducir el segundo número.
                  controller: n2, // Vincula el campo con el controlador n2.
                  decoration: InputDecoration(
                    // Define la apariencia del campo de texto.
                    labelText: "Escribe otro numero", // Muestra la etiqueta del segundo campo.
                    border: OutlineInputBorder(), // Agrega un borde alrededor del campo.
                    prefixIcon: Icon(
                      Icons.amp_stories,
                    ), // Agrega un icono al inicio del campo.
                  ),
                ),
                SizedBox(
                  height: 20,
                ), // Agrega un espacio vertical de 20 píxeles.
                /*Text(r, // Código comentado que anteriormente mostraba el resultado directamente.
                style: TextStyle( // Define el estilo del resultado.
                  fontSize: 24, // Establece el tamaño de la letra.
                  fontWeight: FontWeight.bold, // Hace que el resultado aparezca en negritas.
                  color: Colors.indigo, // Establece el color del resultado.
                ),
                ),*/
                DropdownButtonFormField<String>(
                  // Crea una lista desplegable para seleccionar la operación.
                  decoration: InputDecoration(
                    // Define la apariencia de la lista desplegable.
                    labelText: "Elige la operación", // Muestra la etiqueta de la lista.
                    border: OutlineInputBorder(), // Agrega un borde alrededor de la lista.
                  ),
                  items: [
                    // Contiene las opciones disponibles para seleccionar.
                    DropdownMenuItem(
                      value: 'Suma',
                      child: Text('Suma'),
                    ), // Opción para realizar una suma.
                    DropdownMenuItem(
                      value: 'Resta',
                      child: Text('Resta'),
                    ), // Opción para realizar una resta.
                    DropdownMenuItem(
                      value: 'Multiplicación',
                      child: Text('Multiplicación'),
                    ), // Opción para multiplicar.
                    DropdownMenuItem(
                      value: 'División',
                      child: Text('División'),
                    ), // Opción para dividir.
                  ],
                  onChanged: (value) => oper =
                      value!, // Guarda en oper la operación seleccionada.
                  validator: (v) => v == null
                      ? "Elige una operacion"
                      : null, // Valida que se haya seleccionado una operación.
                ),
                SizedBox(
                  height: 20,
                ), // Agrega un espacio vertical de 20 píxeles.
                SizedBox(
                  // Permite controlar el tamaño del botón.
                  width: double.infinity, // Hace que el botón ocupe todo el ancho disponible.
                  child: ElevatedButton.icon(
                    // Crea un botón elevado con un icono.
                    icon: Icon(Icons.summarize), // Agrega un icono al botón.
                    onPressed: Operaciones, // Ejecuta el método Operaciones al presionar el botón.
                    label: Text(
                      'Realizar operación', // Define el texto que aparece dentro del botón.
                      style: TextStyle(
                        // Define el estilo del texto del botón.
                        fontSize: 24, // Establece el tamaño de la letra.
                        fontWeight: FontWeight
                            .bold, // Hace que el texto aparezca en negritas.
                        color: Colors.indigo, // Establece el color del texto.
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}

```

Ilustración 17 Código practica 4.1



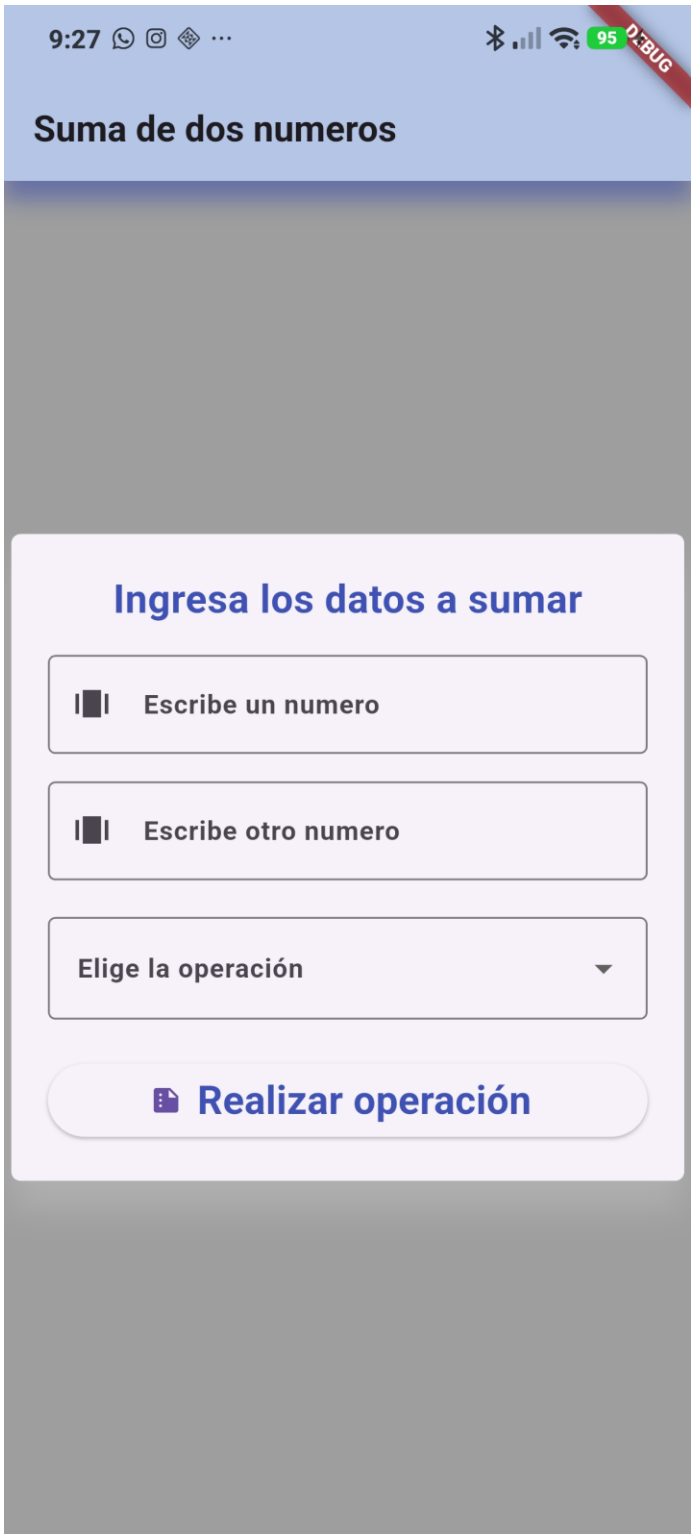


Ilustración 18 Ejecución practica 4.1

Practica 5 “Imágenes”

En esta práctica habilitamos la opción que la aplicación cuente con una carpeta de assets en el cual se guardarán imágenes para poder ser mostradas en la aplicación utilizando opciones de scroll simple para mostrar en 2 filas los 3 tipos de imágenes seleccionadas previamente permitiendo al usuario desplazarse para visualizarlas

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_2/widget/mostrarimagenes.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Mostrarimagenes());
  }
}

```

Ilustración 19 código practica 5

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Flutter.
class Mostrarimagenes extends StatefulWidget { // Declara la pantalla Mostrarimagenes como un StatefulWidget.
  const Mostrarimagenes({super.key}); // Constructor de la clase Mostrarimagenes.
  @override
  State<StatefulWidget> createState() { // Crea el estado que manejará la pantalla.
    return Imagenes(); // Retorna la clase que contiene el diseño de la pantalla.
  }
}
class Imagenes extends State<Mostrarimagenes> { // Define el estado y diseño de la pantalla Mostrarimagenes.
  @override
  Widget build(BuildContext context) { // Construye la interfaz gráfica de la pantalla.
    return Scaffold( // Proporciona la estructura principal de la pantalla.
      backgroundColor: Colors.blue, // Establece el fondo de la pantalla en color azul.
      appBar: AppBar( // Crea la barra superior de la aplicación.
        title: Text('Imagenes', style: TextStyle(color: Colors.black)), // Muestra el título "Imagenes" en color negro.
      ),
      body: Column( // Organiza los elementos principales de forma vertical.
        children: [ // Contiene los elementos que estarán dentro de la columna.
          Expanded( // Hace que este elemento ocupe el espacio disponible proporcionalmente.
            child: Container( // Crea un contenedor para la primera sección.
              width: double.infinity, // Hace que el contenedor ocupe todo el ancho disponible.
              height: 200, // Establece una altura de 200 píxeles.
              child: SingleChildScrollView( // Permite desplazarse cuando el contenido supera el espacio disponible.
                child: Column( // Organiza las imágenes verticalmente.
                  children: [ // Contiene las imágenes que se mostrarán.
                    Image.asset("assets/mapache.jpg", width: 300, height: 300), // Muestra la imagen mapache.jpg desde la carpeta assets.
                    Image.asset("assets/laptop.jpeg", width: 300, height: 300), // Muestra la imagen laptop.jpeg desde la carpeta assets.
                    Image.asset("assets/perro.jpg", width: 300, height: 300), // Muestra la imagen perro.jpg desde la carpeta assets.
                  ],
                ),
              ),
            ),
          ),
          Expanded( // Hace que esta segunda sección ocupe el espacio disponible proporcionalmente.
            child: Container( // Crea un segundo contenedor.
              width: double.infinity, // Hace que el contenedor ocupe todo el ancho disponible.
              height: 200, // Establece una altura de 200 píxeles.
              child: SingleChildScrollView( // Permite desplazarse cuando el contenido supera el espacio disponible.
                child: Column( // Organiza las imágenes verticalmente.
                  children: [ // Contiene las imágenes que se mostrarán.
                    Image.asset("assets/mapache.jpg", width: 300, height: 300), // Muestra nuevamente la imagen del mapache.
                    Image.asset("assets/laptop.jpeg", width: 300, height: 300), // Muestra nuevamente la imagen de la laptop.
                    Image.asset("assets/perro.jpg", width: 300, height: 300), // Muestra nuevamente la imagen del perro.
                  ],
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

Ilustración 20 código practica 5

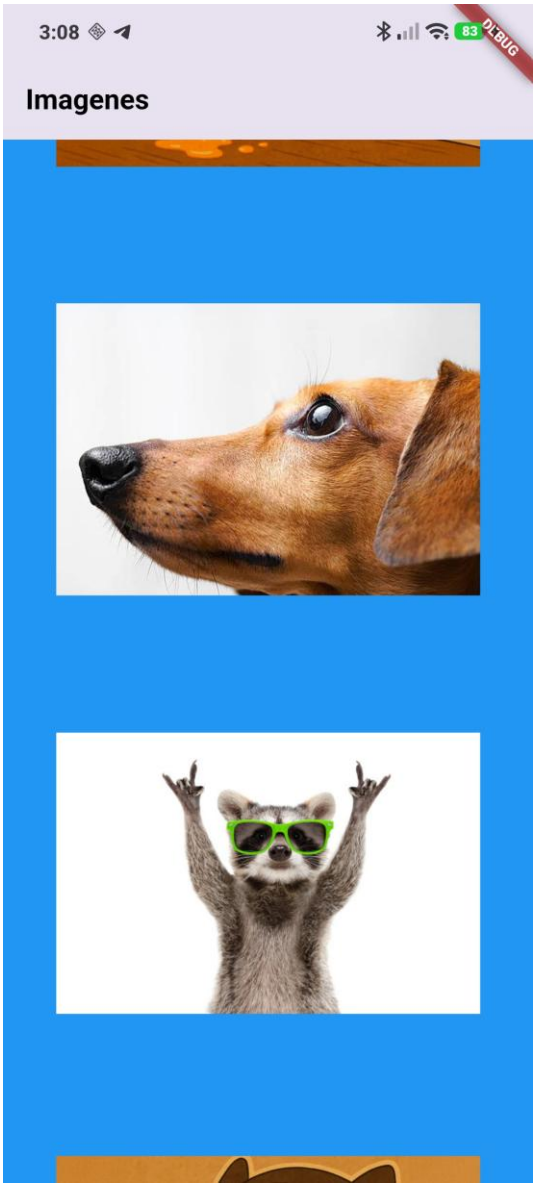


Ilustración 21 ejecución practica 5

Practica 6 “envió de información entre pantallas”

En esta práctica realizamos una ventana de inicio de sesión que permita al usuario ingresar nombre y contraseña para después ser validado y si cumple con las credenciales guardadas será reenviado a otra pantalla de bienvenida en caso que la credenciales sea errónea mostrará un aviso de advertencia

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_3/widget/practica6.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: practica6());
  }
}

```

Ilustración 22 Código practica 6

```dart
import 'package:flutter/material.dart'; // Importa las herramientas visuales de Flutter
import 'package:flutter_application_3/widget/acceso.dart'; // Importa la pantalla Acceso
class practica6 extends StatefulWidget {
  // Crea un StatefulWidget llamado practica6
  const practica6({super.key}); // Constructor de la clase
  @override
  State<StatefulWidget> createState() {
    // Crea el estado de la pantalla
    return login(); // Regresa la clase login que manejará el estado
  }
}
class login extends State<practica6> {
  // Clase que contiene la lógica y diseño de practica6
  TextEditingController usuario = new TextEditingController(); // Controller para obtener el texto del usuario
  TextEditingController password =
      new TextEditingController(); // Controller para obtener la contraseña
  void validar() {
    // Método que valida los datos ingresados
    String u = usuario.text; // Guarda en u el texto escrito en usuario
    String p = password.text; // Guarda en p el texto escrito en password
    setState(() {
      // Indica que habrá un cambio en la pantalla
      if (u == "admin" && p == "12345") {
        // Comprueba si usuario y contraseña son correctos
        //Alertas("Datos correctos", "Bienvenido"); // Mostraría una alerta de datos correctos
        //Navigator.push(//esto nos permite regresar // Navega a otra pantalla pero permite regresar
        Navigator.pushReplacement(
          // Navega a otra pantalla y reemplaza la actual
          //esto no me permite regresar // Al reemplazar la pantalla ya no se puede regresar con el botón atrás
          context, // Indica en qué parte de la aplicación se hará la navegación
          MaterialPageRoute(
            // Crea una ruta para cambiar de pantalla
            builder: (context) => Acceso(
              dato: u,
              datop: p,
            ), // Abre Acceso y manda usuario y contraseña
          ),
        );
      } else {
        // Si los datos no son correctos
        Alertas("Error", "Datos inclorrectos"); // Muestra una alerta de error
        usuario.clear(); // Borra el contenido del campo usuario
        password.clear(); // Borra el contenido del campo contraseña
      }
    });
  }

  void Alertas(String titulo, String mensaje) {
    // Método para mostrar una alerta
    showDialog(
      // Muestra una ventana de diálogo
      context: context, // Indica dónde se mostrará la alerta
      builder: (context) {
        // Construye el contenido de la alerta
        return AlertDialog(
          // Crea la ventana de alerta
          title: Text(
            // Título de la alerta
            titulo, // Utiliza el título recibido por el método
            style: TextStyle(
              fontSize: 25,
              color: Colors.red,
            ), // Cambia tamaño y color del texto
          ),
          content: Text(
            // Contenido de la alerta
            mensaje, // Utiliza el mensaje recibido por el método
            style: TextStyle(
              fontSize: 25,
              color: Colors.red,
            ), // Cambia tamaño y color del texto
          ),
          actions: [
            // Contiene los botones de la alerta
            TextButton(
              // Crea un botón de texto
              onPressed: () {
                // Acción que ocurre al presionar el botón
                Navigator.of(context).pop(); // Cierra la ventana de alerta
              },

```

Ilustración 23 Código practica 6 Código practica 6



```
            child: Text('Aceptar'), // Texto del botón
          ),
        ],
      );
    },
  );
}
@override
Widget build(BuildContext context) {
  // Construye la interfaz gráfica
  return Scaffold(
    // Estructura principal de la pantalla
    backgroundColor: const Color.fromARGB(
      255,
      201,
      201,
      201,
    ), // Color de fondo
    appBar: AppBar(
      // Barra superior
      title: Text('Login'), // Título de la barra
      backgroundColor: const Color.fromARGB(
        255,
        197,
        236,
        236,
      ), // Color de la barra
    ),
    body: Center(
      // Centra el contenido
      child: Padding(
        // Agrega espacio alrededor del contenido
        padding: EdgeInsetsGeometry.all(
          15,
        ), // Agrega 15 de espacio en todos los lados
        child: Card(
          // Crea una tarjeta
          elevation: 8, // Agrega profundidad y sombra
          shadowColor: Colors.black, // Color de la sombra
          shape: RoundedRectangleBorder(
            // Define la forma de la tarjeta
            borderRadius: BorderRadius.circular(15), // Redondea las esquinas
          ),
          child: Padding(
            // Agrega espacio dentro de la tarjeta
            padding: const EdgeInsets.all(
              8.0,
            ), // Agrega 8 de espacio en todos los lados
            child: Column(
              // Organiza los elementos verticalmente
              mainAxisSize: MainAxisSize
                  .min, // La columna ocupa solo el espacio necesario
              children: [
                // Lista de elementos dentro de la columna
                Text(
                  // Muestra texto
                  'Ingresa los datos', // Texto mostrado
                  style: TextStyle(
                    fontSize: 18,
                    color: Colors.blueGrey,
                  ), // Estilo del texto
                ),
                SizedBox(height: 10), // Deja 10 de espacio vertical
                Container(
                  // Contenedor para la imagen
                  width: 120, // Ancho de 120
                  height: 120, // Alto de 120
                  decoration: BoxDecoration(
                    // Permite decorar el contenedor
                    shape: BoxShape.circle, // Hace el contenedor circular
                    color: Colors.white, // Color de fondo blanco
                  ),
                  child: ClipOval(
                    child: Image.asset('assets/usuario.png'),
                  ), // Muestra una imagen recortada en forma circular
                ),
                SizedBox(height: 10), // Deja 10 de espacio vertical
                Container(
                  // Contenedor del campo usuario

```

Ilustración 24 Código practica 6



```
                  width: double.infinity, // Ocupa todo el ancho disponible
                  child: TextField(
                    // Campo para introducir texto
                    controller: usuario, // Conecta el TextField con el controller usuario
                    decoration: InputDecoration(
                      // Configura la apariencia del campo
                      labelText: 'Escribe el usuario', // Texto de etiqueta
                      hintText: 'ingresa el usuario', // Texto de ayuda
                      border: OutlineInputBorder(
                        // Crea un borde alrededor del campo
                        borderRadius: BorderRadius.circular(
                          5,
                        ), // Redondea el borde
                      ),
                    ),
                  ),
                ),
                SizedBox(height: 10), // Deja 10 de espacio vertical
                Container(
                  // Contenedor del campo contraseña
                  width: double.infinity, // Ocupa todo el ancho disponible
                  child: TextField(
                    // Campo para introducir la contraseña
                    controller: password, // Conecta el TextField con el controller password
                    decoration: InputDecoration(
                      // Configura la apariencia del campo
                      labelText: 'Escribe la contraseña', // Texto de etiqueta
                      hintText: 'ingresa la contraseña', // Texto de ayuda
                      border: OutlineInputBorder(
                        // Crea un borde alrededor del campo
                        borderRadius: BorderRadius.circular(
                          5,
                        ), // Redondea el borde
                      ),
                    ),
                  ),
                ),
                SizedBox(height: 10), // Deja 10 de espacio vertical
                Container(
                  // Contenedor del botón
                  width: double.infinity, // Ocupa todo el ancho disponible
                  child: ElevatedButton.icon(
                    // Crea un botón con posibilidad de agregar icono
                    onPressed:
                        validar, // Al presionar ejecuta el método validar
                    label: Text(
                      // Texto del botón
                      'Aceptar', // Texto mostrado
                      style: TextStyle(
                        fontSize: 18,
                        color: Colors.blueGrey,
                      ), // Estilo del texto
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Ilustración 25 Código practica 6



```dart
import 'package:flutter/material.dart'; // Importa las herramientas visuales de Flutter

class Acceso extends StatefulWidget {
  // Crea un StatefulWidget llamado Acceso
  final String dato; // Variable que recibirá el usuario
  final String datop; // Variable que recibirá la contraseña
  const Acceso({
    super.key,
    required this.dato,
    required this.datop,
  }); // Constructor que obliga a recibir dato y datop
  @override
  State<StatefulWidget> createState() {
    // Crea el estado de la pantalla
    return clase(); // Regresa la clase que controla el estado
  }
}

class clase extends State<Acceso> {
  // Clase que contiene el estado y lógica de Acceso
  String d = "", p = ""; // Variables donde se guardarán el usuario y contraseña
  @override
  void initState() {
    // Método que se ejecuta al iniciar la pantalla
    super
        .initState(); // Ejecuta primero el comportamiento original de initState
    d = widget.dato; // Obtiene el dato enviado desde practica6 y lo guarda en d
    p = widget.datop; // Obtiene la contraseña enviada desde practica6 y la guarda en p
  }

  Widget build(BuildContext context) {
    // Construye la interfaz de la pantalla
    return Scaffold(
      // Estructura principal de la pantalla
      backgroundColor: const Color.fromARGB(
        255,
        201,
        201,
        201,
      ), // Color de fondo
      appBar: AppBar(
        // Barra superior
        title: Text('Login'), // Título de la barra
        backgroundColor: const Color.fromARGB(
          255,
          197,
          236,
          236,
        ), // Color de la barra
      ),
      body: Center(
        // Centra el contenido
        child: Padding(
          // Agrega espacio alrededor del contenido
          padding: EdgeInsetsGeometry.all(
            15,
          ), // Agrega 15 de espacio en todos los lados
          child: Card(
            // Crea una tarjeta
            elevation: 8, // Agrega profundidad y sombra
            shadowColor: Colors.black, // Color de la sombra
            shape: RoundedRectangleBorder(
              // Define la forma de la tarjeta
              borderRadius: BorderRadius.circular(15), // Redondea las esquinas
            ),
            child: Padding(
              // Agrega espacio dentro de la tarjeta
              padding: const EdgeInsets.all(8.0), // Agrega 8 de espacio
              child: Column(
                // Organiza los elementos verticalmente
                mainAxisSize: MainAxisSize
                    .min, // La columna ocupa solo el espacio necesario
                children: [
                  // Lista de widgets hijos
                  Text(
                    // Muestra texto
                    'Ingresa los datos', // Texto mostrado
                    style: TextStyle(
                      fontSize: 18,
                      color: Colors.blueGrey,
                    ), // Estilo del texto
                  ),
                  SizedBox(height: 10), // Deja 10 de espacio vertical
                  Container(
                    // Contenedor de la imagen
                    width: 120, // Ancho de 120
                    height: 120, // Alto de 120
                    decoration: BoxDecoration(
                      // Permite decorar el contenedor
                      shape: BoxShape.circle, // Hace circular el contenedor
                      color: Colors.white, // Color de fondo blanco
                    ),
                    child: ClipOval(
                      child: Image.asset('assets/usuario.png'),
                    ), // Muestra la imagen recortada en circulo
                  ),
                  SizedBox(height: 10), // Deja 10 de espacio vertical
                  Text(
                    "Bienvenidos $d \n y su contraseña es $p",
                  ), // Muestra el usuario y contraseña recibidos
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Ilustración 26 Código practica 6



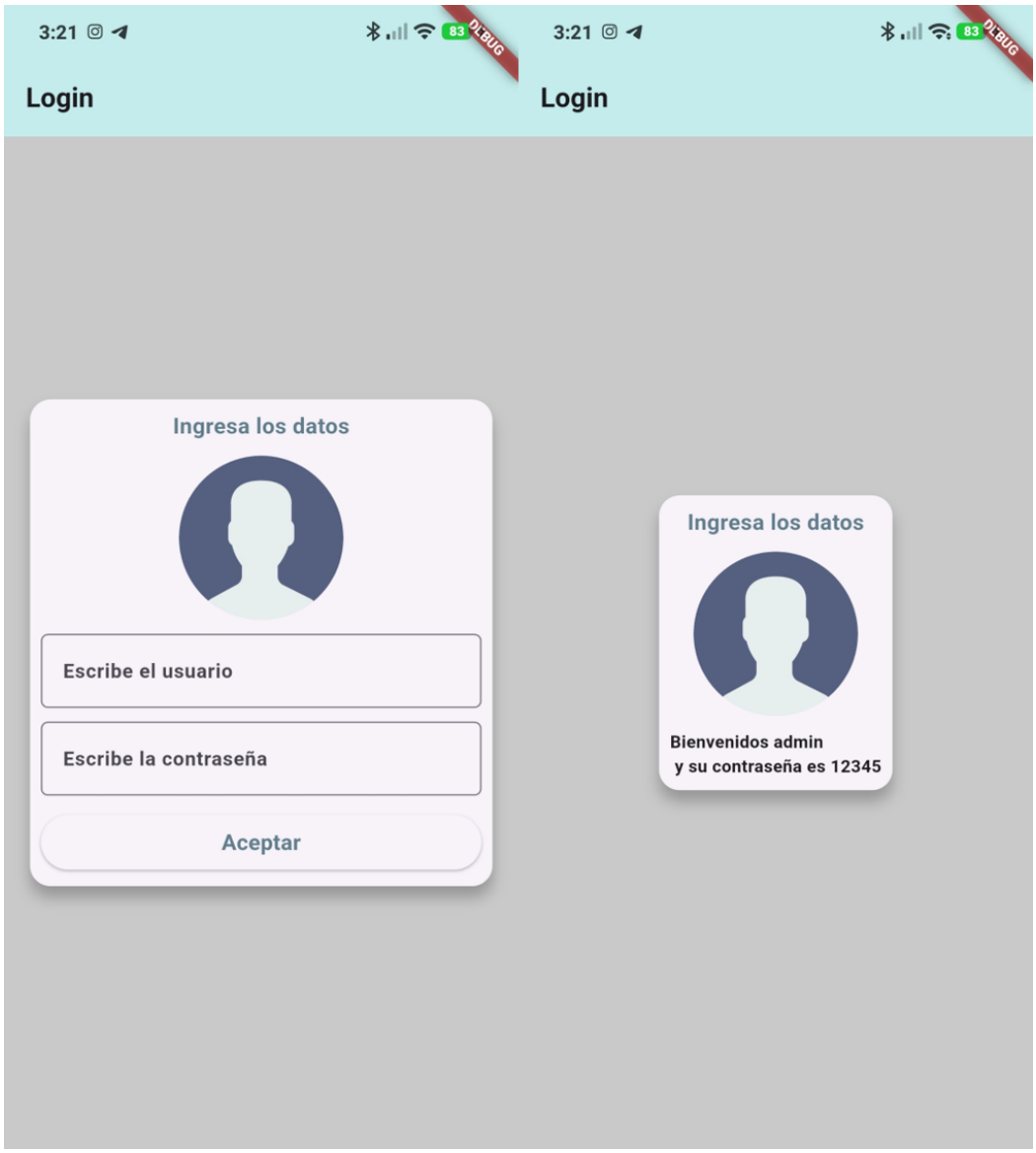


Ilustración 27 Ejecución practica 6

Unidad 2

Practica 7 “Cambio de pantallas y envió de datos”

En esta práctica de flutter implementó una aplicación móvil para validación de un dato y navegación entre pantallas con envío de parámetros

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Material Design
import 'package:flutter_application_1/widget/repaso.dart'; // Importa la pantalla o widget personalizado 'Repaso'

void main() { // Punto de entrada principal de la aplicación
  runApp(const MyApp()); // Inicia y ejecuta el widget raíz de la app
} // Fin de la función main

class MyApp extends StatelessWidget { // Widget sin estado que representa la app
  const MyApp({super.key}); // Constructor constante con clave única para la clave de widget

  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) { // Método que define la estructura visual del widget
    return MaterialApp(home: Repaso()); // Crea la app Material y define 'Repaso' como pantalla inicial
  } // Fin del método build
} // Fin de la clase MyApp
```

Ilustración 28 Código practica 7

```dart
import 'package:flutter/material.dart';

class Pantalla extends StatefulWidget {
  final String dato;
  const Pantalla({super.key, required this.dato});
  @override
  State<StatefulWidget> createState() {
    return metodo();
  }
}

class metodo extends State<Pantalla> {
  String d = "";
  @override
  void initState() {
    super.initState();
    d = widget.dato;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(body: Center(child: Text(d)));
  }
}
```

Ilustración 29 Código practica 7



```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design
import 'package:flutter_application_1/widget/pantalla.dart'; // Importa la pantalla destino 'Pantalla'

class Repaso extends StatefulWidget { // Widget con estado (StatefulWidget) llamado Repaso
  const Repaso({super.key}); // Constructor de Repaso con clave opcional
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() { // Crea el estado asociado al widget
    return Imagenes(); // Devuelve la instancia de la clase que gestiona el estado
  } // Fin de createState
} // Fin de la clase Repaso

class Imagenes extends State<Repaso> { // Clase que gestiona el estado y la UI de Repaso
  final TextEditingController n1 = TextEditingController(); // Controlador para capturar el texto ingresado
  String dato = ""; // Variable para guardar el mensaje descriptivo
  void validar() { // Función para validar los datos ingresados
    setState(() { // Actualiza la UI al cambiar variables de estado
      String t = n1.text; // Guarda el texto escrito en la variable t
      final int d = int.tryParse(n1.text) ?? 0; // Convierte el texto a entero (devuelve 0 si no es número)
      if (t == "") { // Comprueba si el campo de texto está vacío
        dato = "La caja esta vacia"; // Asigna mensaje de caja vacía
      } else { // Si tiene texto
        dato = "La caja no esta vacia"; // Asigna mensaje de caja con texto
      } // Fin del bloque if-else
      if (d == 0) { // Si la conversión fue 0 o texto no numérico
        alertas("Error", "Datos incorrectos"); // Muestra cuadro de diálogo de error
      } else { // Si el valor ingresado es un número válido distinto de 0
        Navigator.pushReplacement( // Reemplaza la pantalla actual navegando a la siguiente
          context, // Contexto de la interfaz
          MaterialPageRoute(builder: (context) => Pantalla(dato: n1.text)), // Transición a Pantalla enviando el dato
        ); // Fin de pushReplacement
      } // Fin del bloque if-else
    }); // Fin de setState
  } // Fin de validar

  void alertas(String titulo, String mensaje) { // Función para mostrar alertas modales
    showDialog( // Despliega un diálogo en pantalla
      context: context, // Contexto para ubicar el diálogo
      builder: (context) { // Constructor que define el contenido del diálogo
        return AlertDialog( // Componente de diálogo modal estándar
          title: Text( // Título del diálogo
            titulo, // Texto del título
            style: TextStyle(fontSize: 25, color: Colors.red), // Estilo: tamaño 25 y color rojo
          ), // Fin de title
          content: Text( // Contenido del diálogo
            mensaje, // Texto del mensaje
            style: TextStyle(fontSize: 25, color: Colors.red), // Estilo: tamaño 25 y color rojo
          ), // Fin de content
          actions: [ // Botones de acción del diálogo
            TextButton( // Botón de texto simple
              onPressed: () { // Evento al presionar el botón
                Navigator.of(context).pop(); // Cierra la alerta modal
              }, // Fin de onPressed
              child: Text('Aceptar'), // Etiqueta del botón
            ), // Fin de TextButton
          ], // Fin de actions
        ); // Fin de AlertDialog
      }, // Fin de builder
    ); // Fin de showDialog
  } // Fin de alertas

  @override // Sobrescribe el método de construcción
  Widget build(BuildContext context) { // Construye el árbol de widgets visuales
    return Scaffold( // Estructura base de pantalla (sin AppBar en este caso)
      body: Column( // Organiza los elementos en una columna vertical
        children: [ // Lista de widgets hijos dentro de la columna
          Expanded( // Hace que el primer bloque ocupe el 50% del espacio vertical disponible
            child: Card( // Tarjeta visual con sombra y bordes redondeados
              child: Container( // Contenedor para aplicar dimensiones
                width: double.infinity, // Ocupa todo el ancho disponible
                child: Padding( // Añade espacio interior alrededor del contenido
                  padding: EdgeInsets.all(20), // Margen interno de 20 pixeles en todos los lados
                  child: Column( // Columna interna para alinear campo y botón
                    mainAxisAlignment: MainAxisAlignment.center, // Centra verticalmente los elementos
                    children: [ // Lista de hijos del formulario
                      TextField(controller: n1), // Campo para entrada de texto conectado a n1
                      SizedBox(height: 20), // Separador transparente de 20px de alto
                      Container( // Contenedor para el botón
                        child: ElevatedButton( // Botón elevado con sombra
                          onPressed: validar, // Ejecuta la función validar al hacer clic
                          child: Text('Enviar'), // Texto dentro del botón
                        ), // Fin de ElevatedButton
                      ), // Fin de Container del botón
                    ], // Fin de children
                  ), // Fin de Column interna
                ), // Fin de Padding
              ), // Fin de Container
            ), // Fin de Card
          ), // Fin de Expanded superior
          Expanded( // Hace que el segundo bloque ocupe el otro 50% del espacio vertical
            child: Container( // Contenedor del mensaje resultante
              child: Column( // Columna para centrar el texto
                mainAxisAlignment: MainAxisAlignment.center, // Centra el texto verticalmente
                children: [ // Lista de hijos del contenedor inferior
                  Text( // Widget que muestra el mensaje guardado en la variable dato
                    dato, // Variable con el texto a mostrar
                    style: TextStyle( // Estilos para personalizar el texto
                      fontSize: 34, // Tamaño de fuente 34px
                      fontFamily: "comics saen", // Nombre de la tipografía personalizada
                      fontWeight: FontWeight.bold, // Texto en negrita
                      color: Colors.blueAccent, // Color del texto azul acentuado
                    ), // Fin de TextStyle
                  ), // Fin de Text
                ], // Fin de children
              ), // Fin de Column
            ), // Fin de Container
          ), // Fin de Expanded inferior
        ], // Fin de children
      ), // Fin de Column principal
    ); // Fin de Scaffold
  } // Fin de build
} // Fin de la clase Imagenes
```

Ilustración 30 Código practica 7



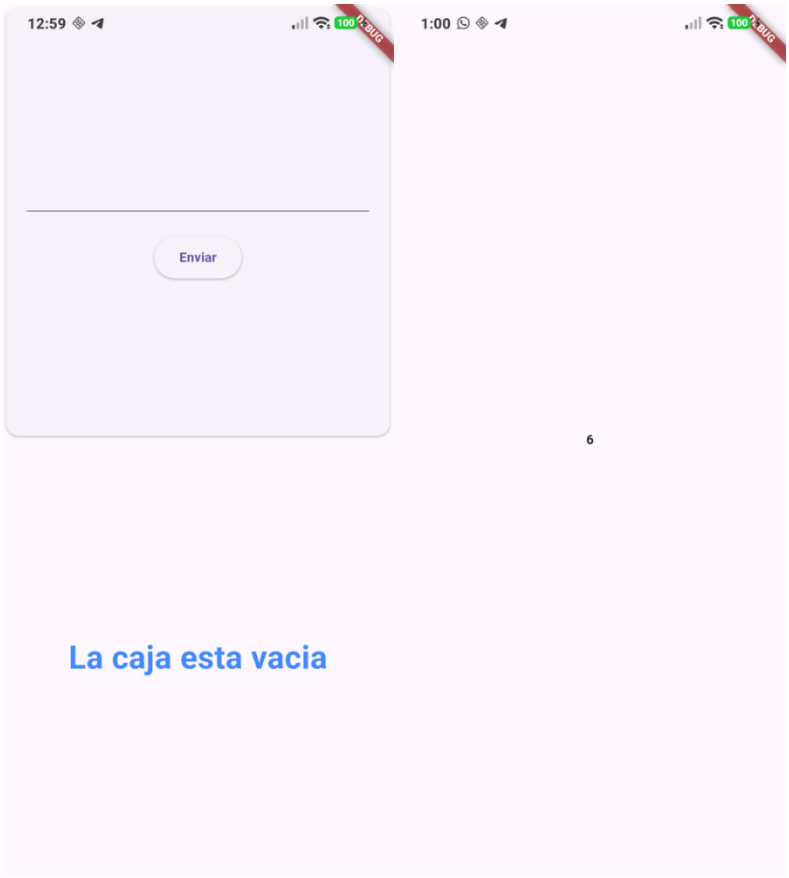


Ilustración 31 Ejecución Practica 7

Practica 8 “Listas y conteo de elementos”

Se desarrolló una aplicación móvil en flutter te funcionaba como una lista de productos interactiva que permite visualizar productos con sus precios y agregar nuevos artículos mediante cajas de texto utiliza validación de datos y muestra mensajes flotantes ante errores. Contando el total de elementos registrados en la barra superior de manera automática y despliega una ventana emergente al presionar un producto

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Material Design
// import 'package:flutter_application_1/widget/lista1.dart'; // Importación comentada (no utilizada)
import 'package:flutter_application_1/widget/lista2.dart'; // Importa la pantalla o widget personalizado 'Lista2'

void main() { // Punto de entrada principal de la aplicación
  runApp(const MyApp()); // Inicia y ejecuta el widget raíz de la app
} // Fin de la función main

class MyApp extends StatelessWidget { // Widget sin estado que representa la app
  const MyApp({super.key}); // Constructor constante con clave única para el widget

  // Este widget es la raiz de la aplicación.
  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) { // Método que define la estructura visual del widget
    return MaterialApp(home: Lista2()); // Crea la app Material y define 'Lista2' como pantalla inicial
  } // Fin del método build
} // Fin de la clase MyApp
```

Ilustración 32 Código Practica 8



```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design

class Lista2 extends StatefulWidget { // Widget con estado (StatefulWidget) llamado Lista2
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() { // Crea el estado asociado al widget
    return Clase2(); // Devuelve la instancia de la clase que gestiona el estado
  } // Fin de createState
} // Fin de la clase Lista2

class Item { // Clase modelo para representar cada producto de la lista
  final String titulo; // Propiedad inmutable para el nombre del producto
  final String subtitulo; // Propiedad inmutable para el precio del producto
  Item(this.titulo, this.subtitulo); // Constructor que inicializa los valores del producto
} // Fin de la clase Item

class Clase2 extends State<Lista2> { // Clase que gestiona el estado y la UI de Lista2
  TextEditingController producto = new TextEditingController(); // Controlador para el campo del nombre
  TextEditingController Precio = new TextEditingController(); // Controlador para el campo del precio

  final List<Item> items = [ // Lista inicial con datos de prueba
    Item('Jabon', '\$120.00'), // Instancia de Item para Jabon
    Item('Agua', '\$10.00'), // Instancia de Item para Agua
    Item('Sal', '\$20.00'), // Instancia de Item para Sal
  ]; // Fin de la lista inicial

  void _MostrarDatos(String nombre, String precioindex) { // Función para abrir un cuadro de diálogo detallado
    print(nombre + precioindex); // Imprime los datos en la consola
    showDialog( // Muestra el diálogo modal en pantalla
      context: context, // Contexto necesario para posicionar la alerta
      builder: (BuildContext context) { // Constructor de la vista del cuadro de diálogo
        return Dialog( // Ventana emergente personalizada
          backgroundColor: Colors.transparent, // Fondo transparente para aplicar bordes redondeados
          child: Container( // Contenedor del contenido del diálogo
            padding: EdgeInsets.all(24), // Margen interno alrededor del diálogo
            decoration: BoxDecoration( // Estilos para la caja flotante
              color: Colors.white, // Fondo blanco para la tarjeta
              borderRadius: BorderRadius.circular(15), // Bordes redondeados de 15px
            ), // Fin de decoration
            child: Column( // Organiza los elementos del diálogo en columna
              mainAxisSize: MainAxisSize.min, // Ajusta el alto al contenido mínimo
              children: [ // Lista de elementos del cuadro de diálogo
                Container( // Contenedor circular para el icono
                  width: 100, // Ancho de 100px
                  height: 100, // Alto de 100px
                  decoration: BoxDecoration( // Estilo del círculo de fondo
                    color: const Color.fromARGB(255, 238, 238, 238), // Color gris claro de fondo
                    shape: BoxShape.circle, // Forma circular
                  ), // Fin de decoration
                  child: Icon(Icons.shopping_bag, color: Colors.blue, size: 40), // Icono de bolsa de compras
                ), // Fin de Container del icono
                SizedBox(height: 10), // Espaciador vertical de 10px
                Text(nombre, style: TextStyle(fontWeight: FontWeight.bold)), // Muestra el nombre en negrita
                SizedBox(height: 10), // Espaciador vertical de 10px
                Text( // Muestra el precio en negrita
                  precioindex, // Texto con el precio
                  style: TextStyle(fontWeight: FontWeight.bold), // Estilo en negrita
                ), // Fin de Text del precio
                // ignore: avoid_unnecessary_containers
                Container( // Contenedor para el botón de cierre
                  child: SizedBox( // Fuerza el ancho completo del botón
                    width: double.infinity, // Ocupa todo el ancho disponible
                    child: ElevatedButton( // Botón para cerrar el diálogo
                      onPressed: () { // Evento al presionar el botón
                        Navigator.pop(context); // Cierra la ventana emergente actual
                      }, // Fin de onPressed
                      child: Text( // Etiqueta del botón
                        'Cerrar', // Texto del botón
                        style: TextStyle(fontWeight: FontWeight.bold), // Texto en negrita
                      ), // Fin de Text
                    ), // Fin de ElevatedButton
                  ), // Fin de SizedBox
                ), // Fin de Container
              ], // Fin de children
            ), // Fin de Column
          ), // Fin de Container
        ); // Fin de Dialog
      }, // Fin de builder
    ); // Fin de showDialog
  } // Fin de _MostrarDatos

  void _agregar() { // Función para añadir un nuevo producto a la lista
    String pd = producto.text; // Guarda el texto del campo producto
    String pc = Precio.text; // Guarda el texto del campo precio
    if (pd.isNotEmpty && pc.isNotEmpty) { // Verifica que ambos campos tengan texto
        setState(() { // Notifica los cambios para redibujar la UI
          items.insert(0, Item(pd, '\$$pc')); // Inserta el nuevo ítem al inicio de la lista
        }); // Fin de setState
        producto.clear(); // Limpia la caja de texto del producto
        Precio.clear(); // Limpia la caja de texto del precio
        ScaffoldMessenger.of(context).showSnackBar( // Muestra una notificación emergente abajo
          SnackBar( // Configuración del mensaje
            content: Text('Numero agregado'), // Texto indicando que se agregó el ítem
            behavior: SnackBarBehavior.floating, // Hace flotar la barra sobre la UI
          ), // Fin de SnackBar
        ); // Fin de showSnackBar
        }else { // Si alguna caja de texto está vacía
      ScaffoldMessenger.of(context).showSnackBar( // Muestra advertencia al usuario
        SnackBar( // Configuración del mensaje de error
          content: Text('Completa los datos'), // Mensaje pidiendo completar los campos
          behavior: SnackBarBehavior.floating, // Hace flotar la barra sobre la UI
        ), // Fin de SnackBar
```

Ilustración 33 Código Practica 8



```
      ); // Fin de showSnackBar
    } // Fin del bloque if-else
  } // Fin de _agregar

  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) { // Método build principal
    return Scaffold( // Estructura base de la pantalla
      appBar: AppBar( // Barra superior de la aplicación
        backgroundColor: Colors.indigoAccent, // Color azul índigo de la barra
        title: Text( // Título principal de la barra
          'Practica de listas', // Texto del título
          style: TextStyle(color: Colors.white), // Texto en color blanco
        ), // Fin de Text
        actions: [ // Acciones o widgets ubicados a la derecha de la barra
          Padding( // Margen para el contador
            padding: EdgeInsets.only(right: 18), // Espacio de 18px a la derecha
            child: Center( // Centra el contador verticalmente
              child: Container( // Caja decorada con la cantidad de productos
                padding: EdgeInsets.symmetric(vertical: 8, horizontal: 12), // Margen interno horizontal y vertical
                decoration: BoxDecoration( // Estilos de la etiqueta contadora
                  color: Colors.white, // Fondo blanco para resaltar
                  borderRadius: BorderRadius.circular(5), // Esquinas redondeadas
                ), // Fin de decoration
                child: Text( // Texto que muestra la cantidad de ítems
                  '${items.length} Productos', // Interpolación con la cantidad total de la lista
                  style: TextStyle(fontWeight: FontWeight.bold), // Texto en negrita
                ), // Fin de Text
              ), // Fin de Container
            ), // Fin de Center
          ), // Fin de Padding
        ], // Fin de actions
      ), // Fin de AppBar
      body: Padding( // Aplica margen externo al cuerpo
        padding: const EdgeInsets.all(8.0), // Margen exterior de 8px
        child: Padding( // Segundo nivel de margen interno
          padding: const EdgeInsets.all(8.0), // Margen interno de 8px
          child: Column( // Organiza el formulario y la lista verticalmente
            children: [ // Elementos dentro del cuerpo principal
              Container( // Contenedor superior para la tarjeta del formulario
                width: double.infinity, // Ocupa todo el ancho
                child: Card( // Tarjeta envolvente para los campos de entrada
                  elevation: 8, // Sombra proyectada por la tarjeta
                  shape: RoundedRectangleBorder( // Define la forma del borde
                    borderRadius: BorderRadiusGeometry.circular(2), // Bordes redondeados leves
                  ), // Fin de shape
                  child: Column( // Organiza las cajas de texto y el botón
                    children: [ // Lista de widgets del formulario
                      TextField( // Campo de entrada para el producto
                        controller: producto, // Conecta con el controlador 'producto'
                        decoration: InputDecoration( // Configuración visual de la entrada
                          label: Text('Escribe el producto'), // Etiqueta flotante del campo
                        ), // Fin de InputDecoration
                      ), // Fin de TextField
                      TextField( // Campo de entrada para el precio
                        controller: Precio, // Conecta con el controlador 'Precio'
                        decoration: InputDecoration( // Configuración visual de la entrada
                          label: Text('Escribe el Precio'), // Etiqueta flotante del campo
                        ), // Fin de InputDecoration
                      ), // Fin de TextField
                      Container( // Contenedor envolvente para el botón
                        width: double.infinity, // Expande el botón horizontalmente
                        child: ElevatedButton.icon( // Botón con ícono
                          onPressed: _agregar, // Ejecuta la función _agregar al hacer clic
                          label: Text('Agregar'), // Etiqueta del botón
                        ), // Fin de ElevatedButton.icon
                      ), // Fin de Container
                    ], // Fin de children del formulario
                  ), // Fin de Column
                ), // Fin de Card
              ), // Fin de Container
              SizedBox(height: 20), // Separador vertical de 20px entre formulario y lista
              Expanded( // Expande el ListView para ocupar el espacio vertical restante
                child: Container( // Contenedor para la lista de productos
                  width: double.infinity, // Ocupa todo el ancho disponible
                  child: ListView.builder( // Constructor dinámico eficiente para listas
                    itemCount: items.length, // Define cuántos elementos se construirán
                    itemBuilder: (context, index) => Card( // Construye una tarjeta para cada posición
                      child: ListTile( // Fila interactiva reutilizable de la lista
                        leading: Icon(Icons.add_alert), // Ícono al inicio de la fila
                        title: Text(items[index].titulo), // Nombre del producto según el índice
                        subtitle: Text(items[index].subtitulo), // Precio del producto según el índice
                        trailing: Icon(Icons.airline_seat_flat_angled_outlined), // Ícono al final de la fila
                        onTap: () { // Evento al tocar la fila entera
                          _MostrarDatos( // Llama a la función para ver los detalles
                            items[index].titulo, // Envía el título actual
                            items[index].subtitulo, // Envía el precio actual
                          ); // Fin de _MostrarDatos
                        }, // Fin de onTap
                      ), // Fin de ListTile
                    ), // Fin de Card
                  ), // Fin de ListView.builder
                ), // Fin de Container
              ), // Fin de Expanded
            ], // Fin de children de la columna principal
          ), // Fin de Column
        ), // Fin de Padding
      ), // Fin de Padding
    ); // Fin de Scaffold
  } // Fin del método build
} // Fin de la clase Clase2
```

Ilustración 34 Código Practica 8



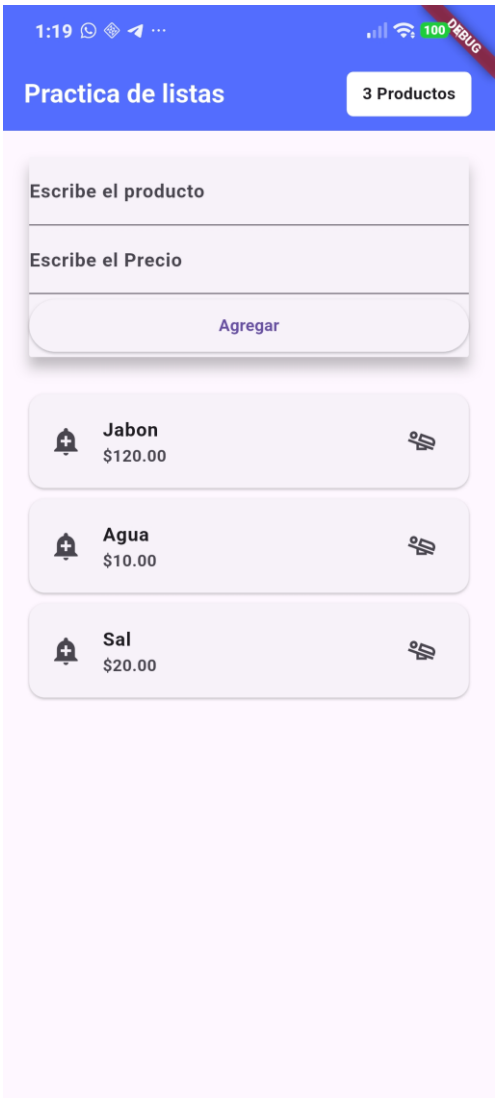


Ilustración 35 Ejecución practica 8

Practica 9 “Listas de números, conteo de números par e impar”

En esta práctica se crea una aplicación móvil en Flutter que permite agregar números a una lista. La app válida que el campo no esté vacío y que el valor ingresado sí sea un número entero. Al agregarlo, revisa si es número par e impar aumentando el contador en cada caso y mostrando un contador general de cuántos números han sido agregados al dar Clic a cada elemento aparece una ventana emergente con su valor.

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Material Design
import 'package:flutter_application_1/widget/Lista.dart'; // Importa la pantalla o widget personalizado 'Lista'

void main() {
  // Punto de entrada principal de la aplicación
  runApp(const MyApp()); // Inicia y ejecuta el widget raíz de la app
} // Fin de la función main

class MyApp extends StatelessWidget {
  // Widget sin estado que representa la app
  const MyApp({
    super.key,
  }); // Constructor constante con clave única para el widget

  // Este widget es la raíz de la aplicación.
  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método que define la estructura visual del widget
    return MaterialApp(
      home: Lista(),
    ); // Crea la app Material y define 'Lista' como pantalla inicial
  } // Fin del método build
} // Fin de la clase MyApp

```

Ilustración 36 Código Practica 9

```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design

class Lista extends StatefulWidget {
  // Widget con estado (StatefulWidget) llamado Lista
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() {
    // Crea el estado asociado al widget
    return Clase2(); // Devuelve la instancia de la clase que gestiona el estado
  } // Fin de createState
} // Fin de la clase Lista
class Item {
  // Clase modelo para representar cada número de la lista
  final String
  num; // Propiedad inmutable para guardar el valor numérico en texto
  Item(this.num); // Constructor que inicializa la propiedad 'num'
} // Fin de la clase Item
class Clase2 extends State<Lista> {
  // Clase que gestiona el estado y la UI de Lista
  int pa = 0; // Contador de números pares
  int inpa = 0; // Contador de números impares
  TextEditingController numero = TextEditingController(); // Controlador para el campo de entrada del número
  final List<Item> items =
      []; // Lista donde se guardan los objetos 'Item' ingresados
  void _AgregarNumero() {
    // Función que procesa y valida la entrada de números
    String n = numero.text; // Guarda el texto ingresado en la variable 'n'
    if (n.isNotEmpty) {
      // Verifica si el campo de texto no está vacío
      if (int.tryParse(n) != null) {
        // Valida si el texto se puede convertir a un entero
        int nu = int.parse(n); // Convierte la cadena de texto a un valor entero
        setState(() {
          // Notifica los cambios para actualizar la interfaz
          if (nu % 2 == 0) {
            // Evalúa si el número es par (residuo 0 al dividir entre 2)
            pa++; // Incrementa en 1 el contador de pares
          } else {
            // Si no es par, es impar
            inpa++; // Incrementa en 1 el contador de impares
          } // Fin de la validación par/impar
          items.insert(
            0,
            Item(n),
          ); // Agrega el nuevo ítem al inicio de la lista
        }); // Fin de setState
        numero.clear(); // Limpia la caja de texto tras agregar el número
      } else {
        // Si la conversión a entero falla (no es un número)
        ScaffoldMessenger.of(context).showSnackBar(
```

Ilustración 37 Código practica 9



```
      ScaffoldMessenger.of(context).showSnackBar(
        // Muestra una alerta informativa emergente
        SnackBar(
          // Configuración del mensaje
          content: Text(
            'No es un numero',
          ), // Texto que indica error de formato // Text
          behavior: SnackBarBehavior.floating, // Flota sobre la pantalla
        ), // Fin de SnackBar // SnackBar
      ); // Fin de showSnackBar
    } // Fin de la validación numérica
  } else {
    // Si la caja de texto estaba vacía
    ScaffoldMessenger.of(context).showSnackBar(
      // Muestra advertencia flotante
      SnackBar(
        // Configuración del mensaje
        content: Text(
          'Agrega un numero',
        ), // Texto solicitando ingresar un número // Text
        behavior: SnackBarBehavior.floating, // Flota sobre la pantalla
      ), // Fin de SnackBar // SnackBar
    ); // Fin de showSnackBar
  } // Fin de la comprobación vacía
} // Fin de la función _AgregarNumero

@override // Sobrescribe el método de construcción de la interfaz
Widget build(BuildContext context) {
  // Método build principal
  return Scaffold(
    // Estructura base de la pantalla
    appBar: AppBar(
      // Barra superior de la aplicación
      backgroundColor: Colors.lightGreen, // Fondo verde claro para la barra
      actions: [
        // Lista de widgets alineados a la derecha
        Padding(
          // Aplica margen alrededor del contador total
          padding: EdgeInsetsGeometry.only(
            right: 18,
          ), // Espacio de 18px a la derecha // EdgeInsetsGeometry.only
          child: Center(
            // Centra verticalmente el contenedor
            child: Container(
              // Caja para el contador total de números
              padding: EdgeInsets.symmetric(
```



Ilustración 38 Código practica 9

```
                padding: EdgeInsets.symmetric(
                  vertical: 8,
                  horizontal: 12,
                ), // Margen interno // EdgeInsets.symmetric
                decoration: BoxDecoration(
                  // Estilo del contador
                  color: Colors.white, // Fondo blanco
                  borderRadius: BorderRadius.circular(5), // Bordes suavizados
                ), // Fin de decoration // BoxDecoration
                child: Text(
                  // Muestra el total de elementos
                  '${items.length} Numeros', // Muestra la cantidad total guardada en 'items'
                  style: TextStyle(
                    fontWeight: FontWeight.bold,
                  ), // Texto en negrita // TextStyle
                ), // Fin de Text // Text
              ), // Fin de Container // Container
            ), // Fin de Center // Center
          ), // Fin de Padding // Padding
        ], // Fin de actions
      ), // Fin de AppBar // AppBar
      body: Padding(
        // Primer nivel de margen alrededor del cuerpo
        padding: EdgeInsets.all(8), // Margen de 8px en todos los bordes
        child: Padding(
          // Segundo nivel de margen interno
          padding: EdgeInsets.all(8), // Margen de 8px
          child: Column(
            // Organiza los elementos verticalmente
            children: [
              // Lista de widgets de la columna principal
              Container(
                // Contenedor para la tarjeta del formulario
                width: double.infinity, // Ocupa todo el ancho horizontal
                child: Card(
                  // Tarjeta visual con sombra para la entrada
                  elevation: 8, // Define el nivel de sombra
                  shape: RoundedRectangleBorder(
                    // Define la forma del borde
                    borderRadius: BorderRadiusGeometry.circular(
                      2,
                    ), // Bordes ligeramente redondeados // BorderRadiusGeometry.circular
                  ), // Fin de shape // RoundedRectangleBorder
                  child: Column(
                    // Organiza el TextField y el botón verticalmente
                    children: [
```

Ilustración 39 Código practica 9



```
                children: [
                  // Elementos dentro de la tarjeta
                  TextField(
                    // Campo de entrada para escribir el número
                    controller: numero, // Conectado al controlador 'numero'
                    decoration: InputDecoration(
                      // Configura la etiqueta del campo
                      label: Text(
                        'Escribe el numero',
                      ), // Etiqueta descriptiva // Text
                    ), // Fin de InputDecoration // InputDecoration
                  ), // Fin de TextField // TextField
                  Container(
                    // Contenedor para expandir el botón
                    width:
                        double.infinity, // Expande el botón horizontalmente
                    child: ElevatedButton(
                      // Botón para ejecutar el guardado
                      onPressed:
                          _AgregarNumero, // Ejecuta la función al presionar
                      child: Text('Agregar'), // Etiqueta del botón
                    ), // Fin de ElevatedButton // ElevatedButton
                  ), // Fin de Container // Container
                ], // Fin de children dentro de la tarjeta
              ), // Fin de Column de la tarjeta // Column
            ), // Fin de Card // Card
          ), // Fin de Container de la tarjeta // Container
          SizedBox(height: 20), // Separador transparente de 20px de alto
          Row(
            // Fila horizontal para mostrar los contadores de pares e impares
            mainAxisAlignment: MainAxisAlignment
                .spaceBetween, // Distribuye los elementos en los extremos
            children: [
              // Elementos de la fila
              Padding(
                // Margen para la caja de pares
                padding: EdgeInsets.only(right: 18), // Margen derecho
                child: Center(
                  // Centra el contenedor
                  child: Container(
                    // Caja que contiene el contador de pares
                    padding: EdgeInsets.symmetric(
                      // Margen interno
                      vertical: 8, // 8px arriba y abajo
                      horizontal: 12, // 12px a los lados
                    ), // Fin de EdgeInsets.symmetric // EdgeInsets.symmetric
```

Ilustración 40 Código practica 9



```
        ), // Fin de EdgeInsets.symmetric // EdgeInsets.symmetric
        decoration: BoxDecoration(
          // Estilos para la caja de pares
          color: Colors.white, // Fondo blanco
          borderRadius: BorderRadius.circular(
            5,
          ), // Bordes redondeados // BorderRadius.circular
        ), // Fin de decoration // BoxDecoration
        child: Text(
          // Muestra el total de números pares
          '${pa} Pares', // Muestra el contador 'pa'
          style: TextStyle(
            fontWeight: FontWeight.bold,
          ), // Texto en negrita // TextStyle
        ), // Fin de Text // Text
      ), // Fin de Container // Container
    ), // Fin de Center // Center
  ), // Fin de Padding de pares // Padding
  Padding(
    // Margen para la caja de impares
    padding: EdgeInsets.only(right: 18), // Margen derecho
    child: Center(
      // Centra el contenedor
      child: Container(
        // Caja que contiene el contador de impares
        padding: EdgeInsets.symmetric(
          // Margen interno
          vertical: 8, // 8px arriba y abajo
          horizontal: 12, // 12px a los lados
        ), // Fin de EdgeInsets.symmetric // EdgeInsets.symmetric
        decoration: BoxDecoration(
          // Estilos para la caja de impares
          color: Colors.white, // Fondo blanco
          borderRadius: BorderRadius.circular(
            5,
          ), // Bordes redondeados // BorderRadius.circular
        ), // Fin de decoration // BoxDecoration
        child: Text(
          // Muestra el total de números impares
          '${inpa} Impares', // Muestra el contador 'inpa'
          style: TextStyle(
            fontWeight: FontWeight.bold,
          ), // Texto en negrita // TextStyle
        ), // Fin de Text // Text
      ), // Fin de Container // Container
    ), // Fin de Center // Center
```

Ilustración 41 Código practica 9

```
230                ), // Fin de Padding de impares // Padding
231              ], // Fin de children de la fila
232            ), // Fin de Row // Row
233            SizedBox(height: 20), // Separador transparente de 20px de alto
234            Expanded(
235              // Hace que el contenedor de la lista ocupe el resto del espacio disponible
236              child: Container(
237                // Contenedor envolvente de la lista
238                width: double.infinity, // Ocupa todo el ancho disponible
239                child: ListView.builder(
240                  // Constructor dinámico eficiente para renderizar la lista
241                  itemCount: items.length, // Define la cantidad total de filas según 'items'
242                  itemBuilder: (context, index) => Card(
243                    // Construye una tarjeta para cada posición
244                    child: ListTile(
245                      // Fila estándar de lista
246                      leading: Icon(
247                        Icons.add_alert,
248                      ), // Icono ubicado al inicio de la fila // Icon
249                      title: Text(
250                        items[index].num,
251                      ), // Muestra el número almacenado en esa posición // Text
252                      trailing: Icon(
253                        Icons.numbers,
254                      ), // Icono de numeral ubicado al final de la fila // Icon
255                    ), // Fin de ListTile // ListTile
256                  ), // Fin de Card // Card
257                ), // Fin de ListView.builder // ListView.builder
258              ), // Fin de Container // Container
259            ), // Fin de Expanded // Expanded
260          ], // Fin de children de la columna principal
261        ), // Fin de Column // Column
262      ), // Fin de Padding interno // Padding
263    ), // Fin de Padding principal // Padding
264  ); // Fin de Scaffold // Scaffold
265  } // Fin del método build
266 } // Fin de la clase Clase2
267
```

Ilustración 42 Código practica 9

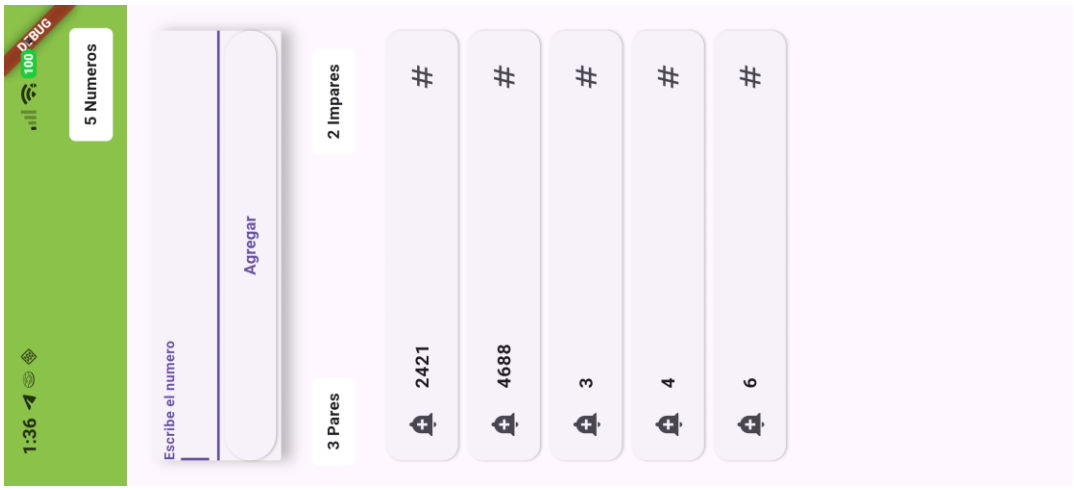


Ilustración 43 Ejecución practica 9

Practica 10 ‘’Login, validación de datos, envió de información y lista’’

En esta práctica se configuro una ventana de inicio de sesión que valida que el nombre y contraseña cumplan con una condición, al hacerlo manda el dato del nombre para ponerlo en la barra de título y por medio de un botón permite agregar contactos de teléfono que se podrán visualizar en la lista.

```dart
import 'package:flutter/material.dart'; // Importa los componentes visuales de Material Design
import 'package:flutter_application_2/widget/login.dart'; // Importa la pantalla o widget personalizado 'Login'

void main() {
  // Punto de entrada principal de la aplicación
  runApp(const MyApp()); // Inicia y ejecuta el widget raíz de la app
} // Fin de la función main

class MyApp extends StatelessWidget {
  // Widget sin estado que representa la app
  const MyApp({
    super.key,
  }); // Constructor constante con clave única para el widget

  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método que define la estructura visual del widget
    return MaterialApp(
      home: Login(),
    ); // Crea la app Material y define 'Login' como pantalla inicial
  } // Fin del método build
} // Fin de la clase MyApp

```

Ilustración 44 Código practica 10

```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design
import 'package:flutter_application_2/widget/Lista.dart'; // Importa la pantalla destino 'Lista'

class Login extends StatefulWidget {
  // Widget con estado (StatefulWidget) llamado Login
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() {
    // Crea el estado asociado al widget
    return inicio(); // Devuelve la instancia de la clase que gestiona el estado
  } // Fin de createState
} // Fin de la clase Login

class inicio extends State<Login> {
  // Clase que gestiona el estado y la UI de Login
  TextEditingController nombre =
      new TextEditingController(); // Controlador para el campo del nombre
  TextEditingController correo =
      new TextEditingController(); // Controlador para el campo del correo
  TextEditingController contra =
      new TextEditingController(); // Controlador para el campo de la contraseña

  void _validar() {
    // Función para validar los datos del formulario antes de navegar
    String n = nombre.text; // Guarda el texto escrito en la caja de nombre
    String c = correo.text; // Guarda el texto escrito en la caja de correo
    String P = contra.text; // Guarda el texto escrito en la caja de contraseña
    if (n.isNotEmpty && c.isNotEmpty && P.isNotEmpty) {
      // Verifica que ningún campo esté vacío
      if (int.tryParse(P) == null) {
        // Comprueba si la contraseña no es un valor numérico
        ScaffoldMessenger.of(context).showSnackBar(
          // Muestra alerta de contraseña no numérica
          SnackBar(
            // Configuración de la barra emergente
            content: Text(
              'La contrasena no es numero',
            ), // Mensaje de advertencia
            behavior: SnackBarBehavior.floating, // Flota sobre la pantalla
          ), // Fin de SnackBar
        ); // Fin de showSnackBar
      } else {
        // Si la contraseña es numérica
        if ((P.length >= 5 && P.length <= 10) && (n.length >= 3)) {
          // Valida longitud de contraseña (5-10) y nombre (>=3)
          Navigator.pushReplacement(
            // Reemplaza la pantalla actual con la nueva pantalla
            context, // Contexto de la interfaz
            MaterialPageRoute(
              builder: (context) => Lista(Nomb: nombre.text),
            ), // Navega a Lista enviando el nombre

```

Ilustración 45Código practica 10

```
          ); // Fin de pushReplacement
        } else {
          // Si las longitudes de texto no cumplen la regla
          ScaffoldMessenger.of(context).showSnackBar(
            // Muestra alerta de revisión de datos
            SnackBar(
              // Configuración de la barra emergente
              content: Text(
                'Revisa los datos',
              ), // Mensaje pidiendo revisar entradas
              behavior: SnackBarBehavior.floating, // Flota sobre la pantalla
            ), // Fin de SnackBar
          ); // Fin de showSnackBar
        } // Fin del bloque if-else de longitud
      } // Fin del bloque if-else de parseo numérico
    } else {
      // Si falta algún campo por llenar
      ScaffoldMessenger.of(context).showSnackBar(
        // Muestra alerta de datos faltantes
        SnackBar(
          // Configuración de la barra emergente
          content: Text('Falta un dato'), // Mensaje indicando campo incompleto
          behavior: SnackBarBehavior.floating, // Flota sobre la pantalla
        ), // Fin de SnackBar
      ); // Fin de showSnackBar
    } // Fin del bloque if-else principal
  } // Fin de la función _validar

  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método build principal
    return Scaffold(
      // Estructura base de la pantalla
      body: Center(
        // Centra la tarjeta horizontal y verticalmente
        child: Padding(
          // Aplica un margen exterior a la tarjeta
          padding: EdgeInsetsGeometry.all(
            15,
          ), // Margen de 15px alrededor de la tarjeta
          child: Card(
            // Tarjeta contenedora con elevación y sombra
            elevation: 8, // Sombra proyectada por la tarjeta
            shadowColor: Colors.black, // Color negro para la sombra
            shape: RoundedRectangleBorder(
              // Define la forma de los bordes
              borderRadius: BorderRadiusGeometry.circular(
                15,
              ), // Bordes redondeados de 15px
            ), // Fin de shape

```

Ilustración 46 Código practica 10

```
child: Padding(
  // Aplica un margen interno a los campos del formulario
  padding: EdgeInsetsGeometry.all(
    8,
  ), // Margen de 8px dentro de la tarjeta
  child: Column(
    // Organiza los elementos del formulario en columna
    mainAxisSize: MainAxisSize
        .min, // Ajusta el alto al contenido mínimo necesario
    children: [
      // Lista de widgets hijos dentro del formulario
      Text(
        // Título del formulario
        'Ingresa tus credenciales', // Texto del encabezado
        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
        ), // Estilo: tamaño 20 y negrita
      ), // Fin de Text
      TextField(
        // Campo de entrada para el nombre
        controller: nombre, // Vincula con el controlador 'nombre'
        decoration: InputDecoration(
          // Configuración visual de la entrada
          labelText: 'Nombre', // Etiqueta flotante del campo
          icon: Icon(Icons.person), // Icono de usuario al inicio
        ), // Fin de InputDecoration
      ), // Fin de TextField nombre
      TextField(
        // Campo de entrada para el correo
        controller: correo, // Vincula con el controlador 'correo'
        decoration: InputDecoration(
          // Configuración visual de la entrada
          labelText: 'Correo', // Etiqueta flotante del campo
          icon: Icon(Icons.email), // Icono de correo al inicio
        ), // Fin de InputDecoration
      ), // Fin de TextField correo
      TextField(
        // Campo de entrada para la contraseña
        controller: contra, // Vincula con el controlador 'contra'
        decoration: InputDecoration(
          // Configuración visual de la entrada
          labelText: 'Contrasena', // Etiqueta flotante del campo
          icon: Icon(
            Icons.password,
          ), // Icono de contraseña al inicio
        ), // Fin de InputDecoration
      ), // Fin de TextField contraseña
      ElevatedButton(
        onPressed: _validar,

```

Ilustración 47 Código practica 10



```
                    child: Text('Validar'),
                  ), // Botón que ejecuta la función _validar
                ], // Fin de children
              ), // Fin de Column
            ), // Fin de Padding interno
          ), // Fin de Card
        ), // Fin de Padding externo
      ), // Fin de Center
    ); // Fin de Scaffold
  } // Fin del método build
} // Fin de la clase inicio

```

Ilustración 48 Código practica 10

```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design

class Lista extends StatefulWidget {
  // Widget con estado (StatefulWidget) llamado Lista
  final String Nomb; // Variable inmutable para recibir el nombre del usuario
  const Lista({
    super.key,
    required this.Nomb,
  }); // Constructor con parámetro Nomb obligatorio
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() {
    // Crea el estado asociado al widget
    return lis(); // Devuelve la instancia de la clase que gestiona el estado
  } // Fin de createState
} // Fin de la clase Lista

class Item {
  // Clase modelo para representar cada contacto
  final String
  name; // Propiedad inmutable para almacenar el nombre del contacto
  final String
  numero; // Propiedad inmutable para almacenar el teléfono del contacto
  Item(this.name, this.numero); // Constructor para inicializar name y numero
} // Fin de la clase Item

class lis extends State<Lista> {
  // Clase que gestiona el estado y la UI de Lista
  String n = ""; // Variable local para almacenar el nombre del usuario
  void initState() {
    // Se ejecuta automáticamente al inicializar el estado del widget
    super.initState(); // Ejecuta la inicialización de la clase base State
    n = widget.Nomb; // Asigna a 'n' el valor recibido en 'Nomb' desde el widget
  } // Fin de initState

  TextEditingController name = new TextEditingController(); // Controlador para la caja de texto del nombre
  TextEditingController phone = new TextEditingController(); // Controlador para la caja de texto del teléfono
  void _agregar() {
    // Función para abrir el diálogo emergente y agregar un contacto
    showDialog(
      // Despliega un cuadro de diálogo en pantalla
      context: context, // Contexto de la interfaz para posicionar el diálogo
      builder: (context) {
        // Constructor que define el contenido del diálogo
        return AlertDialog(
          // Cuadro de diálogo emergente
          actions: [
            // Lista de widgets que conforman el contenido del diálogo
            TextField(
              // Campo de texto para el nombre del contacto
              controller: name, // Conecta con el controlador 'name'

```

Ilustración 49 Código practica 10

```
            decoration: InputDecoration(
              label: Text('Ingresa el nombre'),
            ), // Etiqueta del campo
          ), // Fin de TextField name
          TextField(
            // Campo de texto para el teléfono del contacto
            controller: phone, // Conecta con el controlador 'phone'
            decoration: InputDecoration(
              label: Text('Telefono'),
            ), // Etiqueta del campo
          ), // Fin de TextField phone
          Center(
            // Centra el botón de acción
            child: TextButton(
              // Botón de texto para confirmar la acción
              onPressed: () {
                // Evento al hacer clic en 'Aceptar'
                String nm = name
                    .text; // Guarda el texto escrito en la caja del nombre
                String phn = phone
                    .text; // Guarda el texto escrito en la caja del teléfono
                if (nm.isNotEmpty && phn.isNotEmpty) {
                  // Comprueba que ambas cajas tengan texto
                  setState(() {
                    // Actualiza el estado para reflejar el nuevo ítem en la interfaz
                    items.insert(
                      0,
                      Item(nm, phn),
                    ); // Inserta el nuevo contacto al inicio de la lista
                  }); // Fin de setState
                  name.clear(); // Limpia la caja de texto del nombre
                  phone.clear(); // Limpia la caja de texto del teléfono
                  Navigator.of(context)
                      .pop(); // Cierra la ventana del cuadro de diálogo
                } else {
                  // Si falta por llenar alguno de los dos campos
                  ScaffoldMessenger.of(context).showSnackBar(
                    // Muestra un mensaje emergente
                    SnackBar(
                      // Configuración del mensaje flotante
                      content: Text('Faltan datos'), // Texto de advertencia
                      behavior: SnackBarBehavior
                          .floating, // Posición flotante sobre la UI
                    ), // Fin de SnackBar
                  ); // Fin de showSnackBar
                } // Fin del bloque if-else
              }, // Fin de onPressed
              child: Text('Aceptar'), // Etiqueta del botón
            ), // Fin de TextButton
          ), // Fin de Center
        ], // Fin de actions
      ); // Fin de AlertDialog
    }, // Fin de builder
  ); // Fin de showDialog
} // Fin de la función _agregar

```

Ilustración 50 Código practica 10



```dart
final List<Item> items =
    []; // Lista vacía para almacenar los objetos 'Item' agregados
@override // Sobrescribe el método de construcción de la interfaz
Widget build(BuildContext context) {
  // Método build principal
  return Scaffold(
    // Estructura base de la pantalla
    appBar: AppBar(
      // Barra superior de navegación
      backgroundColor: Colors.blueGrey, // Color de fondo gris azulado
      title: Text(
        n,
      ), // Muestra el nombre del usuario en el título de la barra
      actions: [
        // Acciones ubicadas a la derecha de la barra superior
        Padding(
          // Aplica margen derecho al botón
          padding: EdgeInsets.only(right: 18), // Margen de 18px a la derecha
          child: Center(
            // Centra verticalmente el botón
            child: Container(
              // Contenedor envolvente del botón
              child: ElevatedButton.icon(
                // Botón elevado con un icono
                onPressed:
                    _agregar, // Ejecuta la función _agregar al presionar
                label: Icon(Icons.add), // Muestra el icono de suma/agregar
              ), // Fin de ElevatedButton.icon
            ), // Fin de Container
          ), // Fin de Center
        ), // Fin de Padding
      ], // Fin de actions
    ), // Fin de AppBar
    body: Center(
      // Centra el contenedor de la lista en el cuerpo
      child: Container(
        // Contenedor para el ListView
        child: ListView.builder(
          // Constructor dinámico eficiente para renderizar la lista
          itemCount: items.length, // Define la cantidad total de filas según la lista 'items'
          itemBuilder: (context, index) => Card(
            // Genera una tarjeta para cada posición
            child: ListTile(
              // Estructura estándar de fila con imagen, título y subtítulo
              leading: Image.asset(
                "assets/contacto.png",
              ), // Muestra una imagen local de contacto al inicio
              title: Text(
                items[index].name,
              ), // Muestra el nombre del contacto según el índice
              subtitle: Text(

```

Ilustración 51 Código practica 10

```
                items[index].numero,
              ), // Muestra el teléfono del contacto según el índice
              onTap: () {
                // Evento que se ejecuta al tocar cualquier elemento de la lista
                ScaffoldMessenger.of(context).showSnackBar(
                  // Muestra una barra emergente con información
                  SnackBar(
                    // Configuración del mensaje
                    content: Text(
                      'Indice seleccionado: $index',
                    ), // Muestra la posición tocada
                    behavior: SnackBarBehavior
                        .floating, // Hace flotar el mensaje sobre la UI
                  ), // Fin de SnackBar
                ); // Fin de showSnackBar
              }, // Fin de onTap
            ), // Fin de ListTile
          ), // Fin de Card
        ), // Fin de ListView.builder
      ), // Fin de Container
    ), // Fin de Center
  ); // Fin de Scaffold
 } // Fin del método build
} // Fin de la clase lis

```

Ilustración 52 Código practica 10

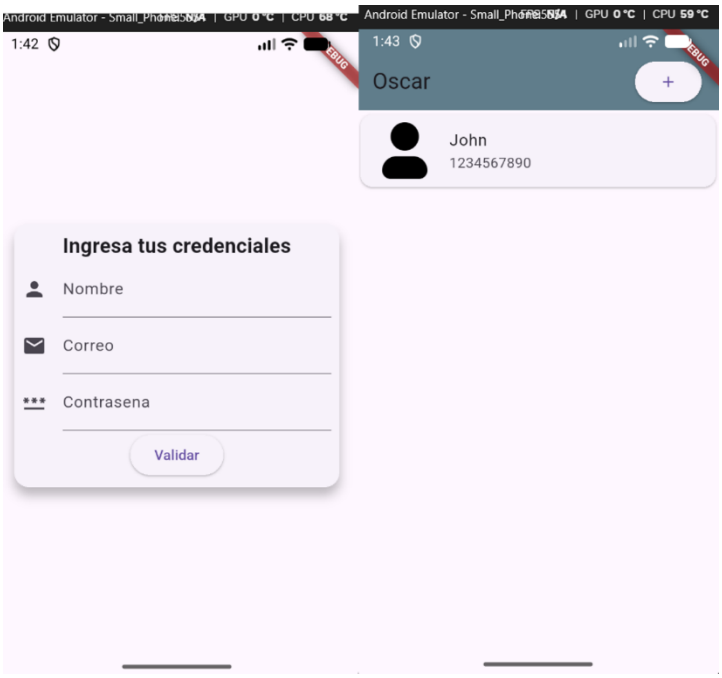


Ilustración 53 Ejecución practica 10

Practica derecho a examen

En esta práctica de flutter se trabaja la captura, validación y paso de datos entre pantallas en Flutter: la primera valida credenciales de acceso, la segunda gestiona un registro dinámico de contactos desde una ventana emergente, y la tercera guarda una lista de productos en un formulario para enviarla a otra pantalla donde se pueden visualizar y eliminar.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_application_3/widget/datos.dart';

void main() {
  // Punto de entrada principal de la aplicación
  runApp(const MyApp()); // Inicia y ejecuta el widget raíz de la app
} // Fin de la función main

class MyApp extends StatelessWidget {
  // Widget sin estado que representa la app
  const MyApp({
    super.key,
  }); // Constructor constante con clave única para el widget

  // Este widget es la raíz de la aplicación.
  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método que define la estructura visual del widget
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Datos(),
    ); // Crea la app Material y define 'Lista' como pantalla inicial
  } // Fin del método build
} // Fin de la clase MyApp

```

Ilustración 54 Código Practica Extra

```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design
import 'package:flutter_application_3/widget/lista.dart'; // Importa la pantalla destino 'Lista'

class Datos extends StatefulWidget {
  // Widget con estado (StatefulWidget) llamado Datos
  const Datos({super.key}); // Constructor de Datos con clave opcional
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() => _DatosState(); // Crea y devuelve el estado asociado '_DatosState'
} // Fin de la clase Datos

class Item {
  // Clase modelo para representar un producto
  final String producto; // Propiedad inmutable para el nombre del producto
  final String descripcion; // Propiedad inmutable para la descripción
  final String precio; // Propiedad inmutable para el precio del producto
  Item(
    this.producto,
    this.descripcion,
    this.precio,
  ); // Constructor para inicializar producto, descripcion y precio
} // Fin de la clase Item

class _DatosState extends State<Datos> {
  // Clase privada que gestiona el estado y la UI de Datos
  TextEditingController pre = TextEditingController(); // Controlador para el campo del nombre del producto
  TextEditingController des =
      TextEditingController(); // Controlador para el campo de la descripción
  TextEditingController prec =
      TextEditingController(); // Controlador para el campo del precio

  List<Item> items =
      []; // Lista vacía para almacenar los objetos 'Item' registrados

  void _Agregar() {
    // Función para validar y agregar un nuevo producto a la lista
    setState(() {
      // Notifica los cambios para redibujar la interfaz de usuario
      String p = pre.text; // Guarda el texto del campo producto en 'p'
      String d = des.text; // Guarda el texto del campo descripción en 'd'
      String pr = prec.text; // Guarda el texto del campo precio en 'pr'
      if (p.isNotEmpty && d.isNotEmpty && pr.isNotEmpty) {
        // Comprueba que los tres campos contengan texto
        items.add(Item(p, d, pr)); // Crea un nuevo objeto 'Item' y lo añade al final de la lista 'items'
        pre.clear(); // Limpia el campo de entrada del producto
        des.clear(); // Limpia el campo de entrada de la descripción
        prec.clear(); // Limpia el campo de entrada del precio
        ScaffoldMessenger.of(
          // Muestra una notificación confirmando la adición
          context, // Contexto de la interfaz de usuario
        ).showSnackBar(
          SnackBar(content: Text('Datos agregados a la lista')),
        ); // Muestra la barra emergente con el texto
```

Ilustración 55 Código Practica Extra

```
      } else {
        // Si falta algún campo por completar
        ScaffoldMessenger.of(
              context,
            ) // Muestra una notificación pidiendo completar los datos
            .showSnackBar(
              SnackBar(content: Text('Faltan datos')),
            ); // Muestra la barra emergente de advertencia
      } // Fin del bloque if-else
    }); // Fin de setState
  } // Fin de la función _Agregar

  void _Enviar() async {
    // Función asíncrona para navegar a la pantalla 'Lista'
    Navigator.push(
      // Navega a una nueva pantalla manteniendo la actual en la pila de navegación
      context, // Contexto de la interfaz de usuario
      MaterialPageRoute(
        builder: (context) => Lista(items: items),
      ), // Pasa la lista 'items' a la pantalla Lista
    ); // Fin de push
    setState(() {}); // Fuerza un redibujado de la pantalla al regresar
  } // Fin de la función _Enviar

  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método build principal
    return Scaffold(
      // Estructura base de la pantalla
      appBar: AppBar(
        // Barra superior de navegación
        title: Text('Datos'), // Título mostrado en la barra superior
        backgroundColor: Colors
            .lightGreen, // Define el color de fondo verde claro para la barra
        actions: [
          IconButton(onPressed: _Enviar, icon: Icon(Icons.list)),
        ], // Botón con icono de lista que llama a _Enviar
      ), // Fin de AppBar
      body: Center(
        // Centra horizontal y verticalmente la tarjeta en la pantalla
        child: Padding(
          // Aplica un margen externo general
          padding: EdgeInsets.all(20), // Margen de 20px en todos los lados
          child: Card(
            // Tarjeta contenedora con sombra y bordes
            elevation: 8, // Define el nivel de elevación y sombra de la tarjeta
            shape: RoundedRectangleBorder(
              // Define los bordes redondeados

```

Ilustración 56 Código Practica Extra

```
              borderRadius: BorderRadiusGeometry.circular(
                15,
              ), // Radio de curvatura de 15px en los bordes
            ), // Fin de shape
            child: Padding(
              // Aplica un margen interno dentro de la tarjeta
              padding: const EdgeInsets.all(15), // Margen interno de 15px
              child: Column(
                // Organiza los campos y botón verticalmente
                mainAxisSize: MainAxisSize
                    .max, // Ocupa el espacio vertical máximo permitido
                children: [
                  // Lista de widgets que conforman el formulario
                  TextField(
                    // Campo de texto para ingresar el nombre del producto
                    controller: pre, // Conecta con el controlador 'pre'
                    decoration: InputDecoration(
                      // Aplica diseño al campo
                      labelText:
                          'Escribe el producto', // Etiqueta flotante del campo
                      border: OutlineInputBorder(), // Añade un borde rectangular cerrado
                    ), // Fin de InputDecoration
                  ), // Fin de TextField producto
                  TextField(
                    // Campo de texto para ingresar la descripción
                    controller: des, // Conecta con el controlador 'des'
                    decoration: InputDecoration(
                      // Aplica diseño al campo
                      labelText: 'Escribe la descripcion', // Etiqueta flotante del campo
                      border: OutlineInputBorder(), // Añade un borde rectangular cerrado
                    ), // Fin de InputDecoration
                  ), // Fin de TextField descripción
                  TextField(
                    // Campo de texto para ingresar el precio
                    controller: prec, // Conecta con el controlador 'prec'
                    decoration: InputDecoration(
                      // Aplica diseño al campo
                      labelText:
                          'Escribe el precio', // Etiqueta flotante del campo
                      border: OutlineInputBorder(), // Añade un borde rectangular cerrado
                    ), // Fin de InputDecoration
                  ), // Fin de TextField precio
                  SizedBox(
                    height: 20,
                  ), // Separador transparente de 20px de alto
                  SizedBox(
                    // Ajusta el tamaño del botón
                    width: double.infinity, // Extiende el botón a todo el ancho disponible
                    child: ElevatedButton.icon(
                      // Botón elevado con icono
                      onPressed:
                          _Agregar, // Ejecuta la función _Agregar al presionar
                      label: Text('Agregar'), // Texto mostrado dentro del botón
                    ), // Fin de ElevatedButton.icon
                  ), // Fin de SizedBox del botón
                ], // Fin de children
              ), // Fin de Column
            ), // Fin de Padding interno
          ), // Fin de Card
        ), // Fin de Padding externo
      ), // Fin de Center
    ); // Fin de Scaffold
  } // Fin del método build
} // Fin de la clase _DatosState
```

Ilustración 57 Código Practica Extra



```dart
import 'package:flutter/material.dart'; // Importa componentes UI de Material Design

import 'datos.dart'; // Importa el archivo local 'datos.dart' para tener acceso a la clase modelo Item

class Lista extends StatefulWidget {
  // Widget con estado (StatefulWidget) llamado Lista
  final List<Item>
  items; // Propiedad inmutable para recibir la lista de productos
  const Lista({
    super.key,
    required this.items,
  }); // Constructor con parámetro obligatorio 'items'
  @override // Sobrescribe el método para vincular la clase de estado
  State<StatefulWidget> createState() => _ListaState(); // Crea y devuelve la instancia de _ListaState
} // Fin de la clase Lista

class _ListaState extends State<Lista> {
  // Clase privada que gestiona el estado y la UI de Lista
  @override // Sobrescribe el método de construcción de la interfaz
  Widget build(BuildContext context) {
    // Método build principal
    final items = widget
        .items; // Referencia local a la lista 'items' recibida desde el widget
    return Scaffold(
      // Estructura base de la pantalla
      appBar: AppBar(
        // Barra superior de navegación
        title: Text('Lista de productos'), // Título de la barra
        backgroundColor: Colors.lightGreen, // Fondo de color verde claro
      ), // Fin de AppBar
      body:
          items
              .isEmpty // Comprueba si la lista de productos está vacía
          ? Center(
              // Si está vacía, muestra un mensaje centrado
              child: Text(
                'Lista vacia',
                style: TextStyle(color: Colors.red),
              ), // Mensaje en color rojo
            ) // Fin del bloque verdadero (lista vacía)
          : Center(
              // Si contiene elementos, muestra la lista centrada
              child: Padding(
                // Aplica un margen exterior a la lista
                padding: EdgeInsets.all(
                  15,
                ), // Margen de 15px en todos los bordes
                child: ListView.builder(
                  // Constructor dinámico eficiente para la lista
                  itemCount:
                      items.length, // Cantidad total de productos a renderizar
                  itemBuilder: (context, index) {
                    // Función que construye cada elemento según su índice

```

Ilustración 58 Código Practica Extra



```
                final item = items[index]; // Obtiene el producto en la posición 'index'
                return Card(
                  // Tarjeta envolvente para cada fila
                  margin: EdgeInsets.all(
                    5,
                  ), // Margen externo de 5px alrededor de la tarjeta
                  child: ListTile(
                    // Fila interactiva con icono, título, subtítulo y botón
                    leading: Icon(
                      Icons.production_quantity_limits,
                    ), // Icono decorativo al inicio
                    title: Text(item.producto), // Nombre del producto
                    subtitle: Column(
                      // Organiza la descripción y precio verticalmente
                      children: [
                        // Widgets hijos del subtítulo
                        Text(item.descripcion), // Descripción del producto
                        Text(
                          'Precio:${item.precio}',
                        ), // Muestra el precio del producto
                      ], // Fin de children de Column
                    ), // Fin de Column
                    trailing: IconButton(
                      // Botón con icono situado al final de la fila
                      onPressed: () {
                        // Evento al presionar el botón eliminar
                        setState(() {
                          // Actualiza el estado y redibuja la pantalla
                          //print('Eliminar $index'); // Comentario con código de pruebas
                          items.removeAt(index); // Elimina el elemento de la lista en la posición 'index'
                        }); // Fin de setState
                      }, // Fin de onPressed
                      icon: Icon(
                        Icons.delete,
                      ), // Icono de papelera de reciclaje
                      color: Colors
                          .red, // Color rojo para el icono de eliminación
                    ), // Fin de IconButton
                  ), // Fin de ListTile
                ); // Fin de Card
              }, // Fin de itemBuilder
            ), // Fin de ListView.builder
          ), // Fin de Padding
        ), // Fin del bloque falso (lista con elementos)
    ); // Fin de Scaffold
  } // Fin del método build
} // Fin de la clase _ListaState

```

Ilustración 59 Código Practica Extra

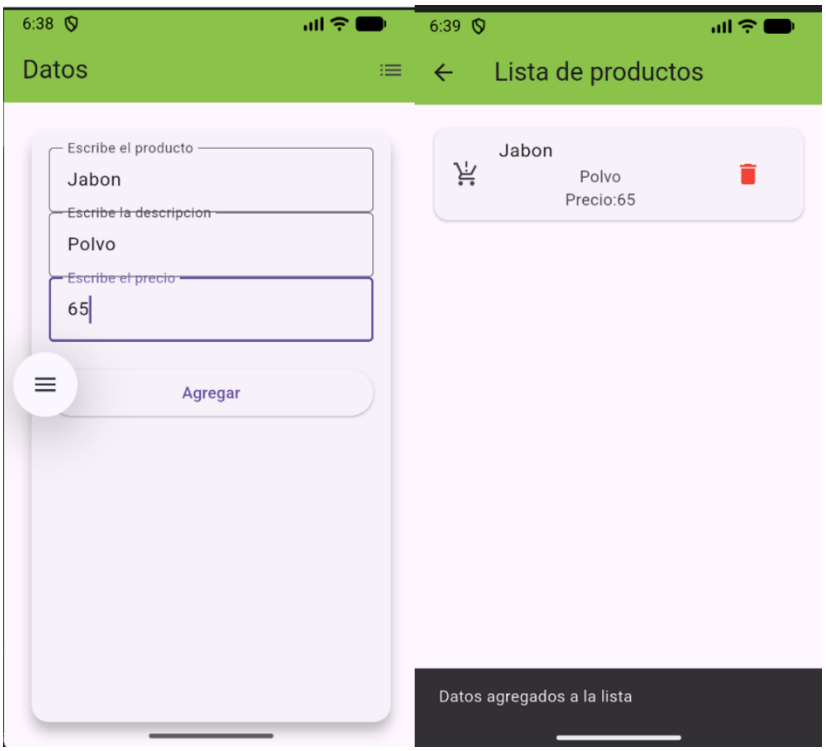


Ilustración 60 Ejecución Practica Extra